Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 14

El Transformer completo

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

En el módulo 13 se construyó la atención causal multicabeza. Este módulo la integra en la arquitectura completa de un modelo de lenguaje: el Transformer *decoder-only*, una pila de L bloques idénticos, cada uno formado por una subcapa de atención y una subcapa de perceptrón multicapa, unidas por conexiones residuales y precedidas por normalizaciones. Se completa con la tabla de embeddings de entrada, la codificación de la posición y la capa de salida que produce los logits. Es la arquitectura publicada de GPT-2 y de la familia LLaMA; Google y xAI han publicado que Gemini 1.5 y Grok-1 son Transformer con mezcla de expertos, y Anthropic no ha publicado la arquitectura de Claude. El módulo termina con la contabilidad de recursos: a partir de cinco números de configuración se calculan exactamente los parámetros, la memoria y el cómputo de entrenamiento de un modelo como LLaMA-2 7B.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. escribir las ecuaciones de un Transformer decoder-only pre-norm, indicar la forma de cada tensor y demostrar que la salida en la posición t solo depende de los tokens $x_{1:t}$;
2. desarrollar el flujo residual como suma de aportaciones, comparar las variantes pre-norm y post-norm y justificar la normalización final;
3. definir las subcapas MLP con GELU y SwiGLU, demostrar las propiedades de la función SiLU y dimensionar d_{ff} para igualar presupuestos de parámetros;
4. construir la codificación sinusoidal y RoPE y demostrar que la primera se desplaza por una aplicación lineal y que la segunda solo depende de la posición relativa;
5. contar exactamente los parámetros de un Transformer, justificar la aproximación $12Ld^2$ y calcular la memoria de inferencia y de entrenamiento;
6. contar los FLOPs de una pasada hacia delante, derivar la regla $C \approx 6ND$ y estimar el tiempo de entrenamiento a partir del rendimiento del hardware.

Requisitos previos

Del módulo 1, los logits, la softmax y la entropía cruzada (temas 4 y 5). De los módulos 2 y 3, el producto de matrices, las matrices ortogonales y las matrices diagonales por bloques. Del módulo 4, la regla de la cadena y el jacobiano. Del módulo 7, el optimizador Adam y sus dos momentos (tema 5). Del módulo 9, el perceptrón multicapa y las funciones de activación. Del módulo 10, la retropropagación en capas lineales. Del módulo 11, las conexiones residuales, RMSNorm, la posición pre-norm y la precisión mixta (temas 2, 3 y 5). Del módulo 13, la atención escalada, la máscara causal y la atención multicabeza, que se usan en todos los temas.

Organización

Tema	Contenido	Fuentes principales
1	Arquitectura del Transformer decoder-only	[17], [12], [8], [1]
2	Flujo residual y posición de la normalización	[19], [12], [4], [6]
3	La subcapa MLP: GELU y SwiGLU	[17], [13], [15], [6]
4	Codificación de la posición: sinusoidal y RoPE	[17], [14], [20], [2]
5	Contar parámetros y memoria	[9], [16], [8], [6]
6	Contar el cómputo: FLOPs y la regla $6ND$	[9], [16], [6]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La arquitectura sigue el capítulo 7 de Jurafsky y Martin [8] (§7.2 a §7.5), el capítulo 12 de Bishop y Bishop [1] (§12.1.7, §12.1.9 y §12.3.1) y las secciones 11.6 y 11.7 de Zhang *et al.* [20], junto con los artículos originales: Vaswani *et al.* [17] para el Transformer y la codificación sinusoidal, Radford *et al.* [12] para GPT-2, Touvron *et al.* [15, 16] para LLaMA y Llama 2, Xiong *et al.* [19] para la posición de la normalización, Zhang y Sennrich [21] para RMSNorm, Shazeer [13] para SwiGLU, Su *et al.* [14] para RoPE, Press y Wolf [11] para los embeddings atados y Kaplan *et al.* [9] para el recuento de parámetros y de cómputo. La descripción del flujo residual como espacio de comunicación procede de Elhage *et al.* [4]. Los ejercicios universitarios proceden de la práctica 1 del curso CS336 de Stanford [6], de la práctica 4 del curso CS224N [2] y de los ejercicios del capítulo 7 de [8]. Las dimensiones de la capa intermedia de LLaMA se han comprobado en el código publicado por Meta [10]. Sobre Claude, Gemini y Grok solo se usa información pública [5, 18].

Notación

Símbolo	Significado
$\mathcal{V}$, $V = \lvert\mathcal{V}\rvert$	vocabulario y su tamaño
n, n_{ctx}	longitud de la secuencia y longitud máxima de contexto
d, h, $d_k = d/h$	dimensión del modelo, cabezas y dimensión por cabeza
L, d_{ff}	número de bloques y dimensión de la capa intermedia del MLP
$H^\ell \in \mathbb{R}^{n\times d}$, h_t^ℓ	flujo residual tras el bloque ℓ y su fila t
$W_E \in \mathbb{R}^{V\times d}$, $W_U \in \mathbb{R}^{d\times V}$	matrices de embedding y de *unembedding*
W_Q, W_K, W_V, W_O	proyecciones de la atención ($d \times d$)
$W_g, W_u \in \mathbb{R}^{d\times d_{\text{ff}}}$	matrices de puerta y de subida de SwiGLU
$W_{\text{d}} \in \mathbb{R}^{d_{\text{ff}}\times d}$	matriz de bajada de SwiGLU
$M \in \{0, -\infty\}^{n\times n}$	máscara causal
$\sigma(x)$, $\text{SiLU}(x) = x\,\sigma(x)$	sigmoide logística y función SiLU
$R(\alpha)$, R_m	rotación de ángulo α; matriz de RoPE en la posición m
$\text{PE}(p)$	codificación sinusoidal de la posición p
N, D, C	número de parámetros, de tokens de entrenamiento y de FLOPs
P	matriz de permutación

Los vectores de tokens son filas: una secuencia es una matriz $X \in \mathbb{R}^{n\times d}$ y una capa lineal actúa como $X \mapsto XW$, como en [8] y en el código de PyTorch. Los números grandes se escriben con espacio fino como separador de miles (6 738 415 616).

Índice

Presentación del módulo **ii**

1 Arquitectura del Transformer decoder-only **1**

1.1 Entrada: tokens y embeddings . . . 1

1.2 El bloque decoder-only . . . 1

1.3 Causalidad . . . 3

1.4 Salida: unembedding y embeddings atados . . . 4

Ejercicios del tema 1 . . . 5

2 Flujo residual y posición de la normalización **6**

2.1 El flujo residual . . . 6

2.2 Pre-norm frente a post-norm . . . 7

2.3 El precio de pre-norm: crecimiento del flujo . . . 8

Ejercicios del tema 2 . . . 9

3 La subcapa MLP: GELU y SwiGLU **10**

3.1 El MLP posición a posición . . . 10

3.2 Activaciones suaves: GELU y SiLU . . . 10

3.3 Unidades lineales con puerta y SwiGLU . . . 11

Ejercicios del tema 3 . . . 12

4 Codificación de la posición: sinusoidal y RoPE **14**

4.1 La atención no ve el orden . . . 14

4.2 Codificación sinusoidal y posiciones aprendidas . . . 14

4.3 Codificación rotatoria (RoPE) . . . 15

Ejercicios del tema 4 . . . 17

5 Contar parámetros y memoria **19**

5.1 Parámetros de un bloque . . . 19

5.2 Parámetros del modelo completo . . . 19

5.3 Memoria . . . 20

Ejercicios del tema 5 . . . 21

6 Contar el cómputo: FLOPs y la regla $6ND$ **23**
6.1 El coste de un producto de matrices . . . 23
6.2 La pasada hacia delante . . . 23
6.3 La pasada hacia atrás y la regla $6ND$ . . . 24
Ejercicios del tema 6 . . . 25

A Soluciones de los ejercicios **27**
A.1 Tema 1 . . . 27
A.2 Tema 2 . . . 29
A.3 Tema 3 . . . 32
A.4 Tema 4 . . . 34
A.5 Tema 5 . . . 37
A.6 Tema 6 . . . 40

Bibliografía **44**

Tema 1

Arquitectura del Transformer decoder-only

Este tema describe la arquitectura completa de un Transformer decoder-only: la entrada (tokens y embeddings), la pila de bloques y la capa de salida que produce los logits. Se demuestran dos propiedades estructurales: cada bloque conserva la forma $n \times d$ y la salida en la posición t solo depende de los tokens $x_{1:t}$. Se sigue [8, §7.2–§7.5], [1, §12.1.7 y §12.3.1], el artículo original [17, §3] y la descripción de la práctica 1 de CS336 [6, §3.1 y §3.4].

1.1 Entrada: tokens y embeddings

Un tokenizador (módulo 12) transforma el texto en una secuencia de enteros $x_{1:n} = (x_1, \dots, x_n)$ con $x_t \in \{1, \dots, V\}$.

Definición 1.1 (Embedding de tokens). Sea $W_E \in \mathbb{R}^{V \times d}$ una matriz de parámetros cuya fila j, $e_j \in \mathbb{R}^d$, es el *embedding* del token j. La entrada de la red es la matriz $H^0 \in \mathbb{R}^{n \times d}$ cuya fila t es

$$h_t^0 = e_{x_t} \ (+\, p_t),$$

donde $p_t \in \mathbb{R}^d$ es un vector de posición que solo se suma en los modelos con posiciones absolutas (tema 4). Equivalentemente, $H^0 = OW_E$, siendo $O \in \{0,1\}^{n \times V}$ la matriz cuyas filas son las codificaciones *one-hot* de los tokens.

La forma matricial OW_E muestra que el embedding es una capa lineal sin sesgo aplicada a vectores *one-hot*; en la práctica se implementa como una indexación de filas, que no hace ninguna multiplicación [8, §7.4].

1.2 El bloque decoder-only

Se recuerda la atención del módulo 13. Para una matriz $Z \in \mathbb{R}^{n \times d}$ y h cabezas de dimensión $d_k = d/h$, la cabeza i calcula $Q_i = ZW_Q^{(i)}$, $K_i = ZW_K^{(i)}$, $V_i = ZW_V^{(i)}$, con $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d_k}$, y

$$\mathrm{head}_i(Z) = \mathrm{softmax}\Big(\frac{Q_i K_i^\top}{\sqrt{d_k}} + M\Big) V_i, \qquad M_{tu} = \begin{cases} 0, & u \le t, \\ -\infty, & u > t, \end{cases}$$

donde la softmax se aplica por filas. La atención multicabeza concatena las cabezas y proyecta: $\mathrm{MHA}(Z) = [\mathrm{head}_1(Z) \ \cdots \ \mathrm{head}_h(Z)]\, W_O$, con $W_O \in \mathbb{R}^{d \times d}$. Agrupando por columnas las h matrices $W_Q^{(i)}$ se obtiene una sola matriz $W_Q \in \mathbb{R}^{d \times d}$, y lo mismo para W_K y W_V.

Definición 1.2 (Bloque pre-norm). Sean $\mathrm{Norm}_1, \mathrm{Norm}_2 : \mathbb{R}^d \to \mathbb{R}^d$ dos normalizaciones (RMSNorm en LLaMA, LayerNorm en GPT-2), aplicadas fila a fila, y sea $\mathrm{MLP} : \mathbb{R}^d \to \mathbb{R}^d$

un perceptrón aplicado fila a fila (tema 3). El *bloque decoder-only pre-norm* es la aplicación $B : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$, $H \mapsto H'$, dada por

$$A = H + \mathrm{MHA}\big(\mathrm{Norm}_1(H)\big), \qquad H' = A + \mathrm{MLP}\big(\mathrm{Norm}_2(A)\big).$$

Definición 1.3 (Transformer decoder-only). Un *Transformer decoder-only* de L capas es la composición

$$H^\ell = B_\ell\big(H^{\ell-1}\big) \quad (\ell = 1, \dots, L), \qquad Z = \mathrm{Norm}_f\big(H^L\big)\, W_U \in \mathbb{R}^{n\times V},$$

donde H^0 es la entrada de la definición 1.1, cada B_ℓ es un bloque con parámetros propios, Norm_f es la *normalización final* y $W_U \in \mathbb{R}^{d\times V}$ es la matriz de *unembedding*. La fila t de Z son los logits de la distribución del token siguiente: $P(x_{t+1} = j \mid x_{1:t}) = \mathrm{softmax}(z_t)_j$.

La figura 1.1 representa la arquitectura. Las dos subcapas tienen papeles complementarios: la atención es la única operación que combina filas distintas, es decir, que mezcla información *entre* tokens; las normalizaciones y el MLP actúan sobre cada fila por separado [8, §7.2].

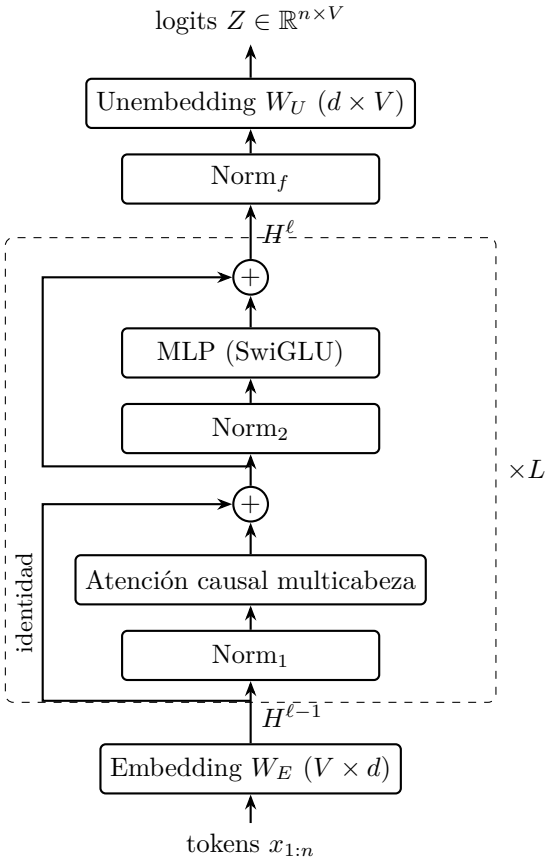


Figura 1.1: Transformer decoder-only pre-norm. El recuadro discontinuo es un bloque (definición 1.2), que se repite L veces. Las líneas de la izquierda son las conexiones residuales: el camino desde el embedding hasta Norm_f solo atraviesa sumas.

Proposición 1.4 (Conservación de la forma). *Si $H \in \mathbb{R}^{n\times d}$, entonces $B(H) \in \mathbb{R}^{n\times d}$. En consecuencia, todos los flujos $H^0, \dots, H^L$ tienen forma $n \times d$, y solo la entrada (n enteros $\to n \times d$) y la salida ($n \times d \to n \times V$) cambian de dimensión.*

Demostración. Las normalizaciones actúan fila a fila de $\mathbb{R}^d$ en $\mathbb{R}^d$. Cada cabeza produce una matriz $n \times d_k$; la concatenación de las h cabezas tiene forma $n \times hd_k = n \times d$, y el producto por $W_O \in \mathbb{R}^{d\times d}$ conserva esa forma. El MLP transforma cada fila de $\mathbb{R}^d$ en $\mathbb{R}^d$. Las dos sumas residuales suman matrices $n \times d$. Por inducción sobre ℓ, todo H^ℓ es $n \times d$. ■

La proposición 1.4 es la razón por la que los bloques pueden apilarse en cualquier número: L es un hiperparámetro libre, que va de 12 en GPT-2 *small* a 80 en Llama 2 70B [12, 16].

1.3 Causalidad

Proposición 1.5 (Causalidad del decoder). *En un Transformer decoder-only, la fila t de H^ℓ, para todo ℓ, y la fila t de los logits Z solo dependen de los tokens $x_1, \ldots, x_t$.*

Demostración. Se dice que una matriz H *es causal* si cada fila h_t es función de $x_{1:t}$. Se razona por inducción sobre ℓ. La matriz H^0 es causal porque $h_t^0 = e_{x_t} + p_t$ solo depende de x_t. Supóngase que $H^{\ell-1}$ es causal.

(i) Toda operación aplicada fila a fila (Norm_1, Norm_2, MLP, las proyecciones ZW) transforma matrices causales en matrices causales, porque la fila t del resultado solo depende de la fila t de la entrada.

(ii) En cada cabeza, los pesos de la fila t son $\alpha_{tu} = \mathrm{softmax}(s_t + M_t)_u$, con $s_{tu} = q_t \cdot k_u/\sqrt{d_k}$. Para $u > t$, el exponente es $-\infty$ y $\alpha_{tu} = e^{-\infty}/\sum_{u'} e^{s_{tu'}+M_{tu'}} = 0$; para $u \le t$, α_{tu} solo depende de q_t y de $k_1, \ldots, k_t$. La salida $\sum_{u\le t} \alpha_{tu} v_u$ solo depende de las filas $1, \ldots, t$ de una matriz causal, luego de $x_{1:t}$.

(iii) La suma de matrices causales es causal.

Así A y $H^\ell = B_\ell(H^{\ell-1})$ son causales. Finalmente, $Z = \mathrm{Norm}_f(H^L)W_U$ se obtiene con operaciones fila a fila. ■

Corolario 1.6 (Una pasada, n predicciones). *Una sola evaluación del Transformer sobre $x_{1:n}$ proporciona las n distribuciones $P(\cdot \mid x_{1:t})$, $t = 1, \ldots, n$. La pérdida de entrenamiento de una secuencia, la entropía cruzada media del módulo 1,*

$$\mathcal{L} = -\frac{1}{n-1}\sum_{t=1}^{n-1} \ln \mathrm{softmax}(z_t)_{x_{t+1}},$$

se calcula, por tanto, con una única pasada hacia delante y una única pasada hacia atrás.

Demostración. Por la proposición 1.5, la fila z_t calculada con toda la secuencia coincide con la que se obtendría con la secuencia truncada $x_{1:t}$. Es, por tanto, el vector de logits de $P(\cdot \mid x_{1:t})$ de la definición 1.3. ■

Esta propiedad es la que hace eficiente el entrenamiento de los Transformer frente a las redes recurrentes: las n posiciones se procesan en paralelo como un producto de matrices [8, §7.3]. En la generación, en cambio, los tokens se producen de uno en uno, y la proposición 1.5 permite reutilizar las claves y los valores ya calculados (la caché KV del módulo 19).

1.4 Salida: unembedding y embeddings atados

La capa de salida transforma cada vector $h \in \mathbb{R}^d$ en V logits mediante $z = hW_U$. Si se escribe $u_j \in \mathbb{R}^d$ para la columna j de W_U, el logit del token j es el producto escalar $z_j = h \cdot u_j$: el modelo puntúa cada token por la alineación de h con un vector *plantilla* de ese token [8, §7.5].

Definición 1.7 (Embeddings atados). Un Transformer tiene los *embeddings atados* (*weight tying*) si $W_U = W_E^\top$, es decir, si la plantilla de salida de cada token coincide con su embedding de entrada: $z_j = h \cdot e_j$.

Press y Wolf [11] mostraron que atar los embeddings reduce la perplejidad de varios modelos de lenguaje y el número de parámetros. El Transformer original comparte la misma matriz entre los embeddings y la capa previa a la softmax, y multiplica los embeddings por $\sqrt{d}$ [17, §3.4]. GPT-2 también ata los embeddings; LLaMA y Llama 2 usan dos matrices independientes [10].

Proposición 1.8 (Ahorro del atado). *Atar los embeddings reduce el número de parámetros en exactamente Vd. Para un vector h fijo, el token de mayor logit es el j que maximiza $\|e_j\| \cos \angle(h, e_j)$.*

Demostración. Sin atar, W_E y W_U aportan Vd parámetros cada una; atadas, se comparte una sola matriz de Vd parámetros. Por otra parte, $z_j = h \cdot e_j = \|h\|\|e_j\| \cos \angle(h, e_j)$, y el factor $\|h\| > 0$ es común a todos los j. ∎

Cuadro 1.1: Elecciones de arquitectura de tres modelos con detalles públicos.

Elemento	Transformer (2017)	GPT-2 (2019)	LLaMA / Llama 2 (2023)
Normalización	LayerNorm, post-norm	LayerNorm, pre-norm	RMSNorm, pre-norm
MLP	ReLU, $d_{\text{ff}} = 4d$	GELU, $d_{\text{ff}} = 4d$	SwiGLU, $d_{\text{ff}} \approx \frac{8}{3}d$
Posición	sinusoidal (sumada)	aprendida (sumada)	RoPE (en q y k)
Sesgos	sí	sí	no
Embeddings	atados	atados	sin atar

Observación 1.9 (Conexión con los LLM). La cuadro 1.1 resume las diferencias entre las tres arquitecturas de referencia [17, 12, 15]. Mistral, Qwen y Gemma siguen el esquema de LLaMA con variaciones. Gemini 1.5 y Grok-1 sustituyen el MLP de cada bloque por una mezcla de expertos (módulo 18) [5, 18]; Anthropic no ha publicado la arquitectura detallada de Claude.

Lecturas recomendadas para este tema

- Jurafsky y Martin [8], §7.2 «Transformer Blocks», §7.3, §7.4 y §7.5 «The Language Modeling Head».
- Bishop y Bishop [1], §12.1.7 «Transformer layers» y §12.3.1 «Decoder transformers».
- Vaswani *et al.* [17], §3.1 y §3.4 «Embeddings and Softmax».
- Hashimoto y Liang [6], §3.1 «Transformer LM» y §3.5 «The Full Transformer LM».

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Un Transformer decoder-only tiene $V = 100$, $d = 8$, $h = 2$, $d_{\text{ff}} = 32$ y recibe una secuencia de $n = 5$ tokens. Indique la forma de: (a) la entrada; (b) H^0; (c) Q_i en una cabeza; (d) la matriz de pesos de atención de una cabeza; (e) la capa intermedia del MLP; (f) los logits. ¿Cuántas distribuciones del token siguiente produce una pasada?

Ejercicio 1.2 ⋆ En un modelo con embeddings atados, ¿qué matriz se comparte y con qué forma? ¿Por qué es razonable que la entrada y la salida usen los mismos vectores? ¿Cuántos parámetros ahorra en GPT-2 *small* ($V = 50\,257$, $d = 768$)?

Ejercicio 1.3 ⋆⋆ Un modelo con embeddings atados tiene $V = 3$ y $d = 2$, con $e_1 = (1, 0)$, $e_2 = (0, 1)$ y $e_3 = (1, 1)$. El vector normalizado de la última posición es $h = (2, 1)$. Calcule los logits y la distribución del token siguiente. ¿Qué token es el más probable y por qué, según la proposición 1.8?

Ejercicio 1.4 ⋆⋆ (a) Con $n = 4$, ¿cuántos pesos de atención de una cabeza pueden ser no nulos? ¿Qué fracción de la matriz $n \times n$ representan cuando $n = 1024$? (b) En la fila $t = 3$ las puntuaciones escaladas son $s_3 = (1, 2, 0, 5)$. Calcule los pesos de atención con la máscara causal.

Ejercicio 1.5 ⋆ ¿Por qué la máscara causal usa $-\infty$ y no 0 en las posiciones futuras? Compruébelo calculando los pesos de la fila del ejercicio 1.4 si se sumara 0 en lugar de $-\infty$.

Fuente: adaptado de Jurafsky y Martin [8], cap. 7, ejercicio 7.2.

Ejercicio 1.6 ⋆⋆⋆ Sea $M \in \{0, -\infty\}^{n \times n}$ la máscara causal y sea P la matriz de la permutación π de $\{1, \dots, n\}$, es decir, $(PA)_{t\cdot} = A_{\pi(t)\cdot}$. Demuestre que $PMP^\top = M$ si y solo si π es la identidad. Interprete el resultado: ¿qué información sobre el orden aporta la máscara por sí sola?

Tema 2

Flujo residual y posición de la normalización

En el módulo 11 se estudiaron las conexiones residuales y la normalización para una red genérica. Este tema las aplica al bloque del tema 1: desarrolla el flujo residual de un Transformer como suma de $2L$ aportaciones, compara las variantes pre-norm y post-norm y justifica la normalización final y la inicialización escalada. Se sigue [8, §7.2], la discusión de la práctica 1 de CS336 [6, §3.4], el análisis de Xiong *et al.* [19], la §2.3 de [12] y la interpretación de Elhage *et al.* [4].

2.1 El flujo residual

Para simplificar la notación, se numeran las $2L$ subcapas del Transformer en el orden en que se aplican: la subcapa $k = 2\ell - 1$ es la atención del bloque ℓ y la subcapa $k = 2\ell$ es su MLP. Sea F_k la función que calcula la subcapa k *incluida su normalización de entrada*: $F_{2\ell-1} = \mathrm{MHA} \circ \mathrm{Norm}_1$ y $F_{2\ell} = \mathrm{MLP} \circ \mathrm{Norm}_2$.

Definición 2.1 (Flujo residual). El *flujo residual* es la sucesión de matrices $R^0 = H^0$ y $R^k = R^{k-1} + F_k(R^{k-1})$, $k = 1, \ldots, 2L$. Se cumple $R^{2\ell} = H^\ell$ y $R^{2\ell-1} = A^\ell$, la matriz intermedia del bloque ℓ. La fila t de R^k es el vector del flujo del token t.

Proposición 2.2 (Desarrollo telescópico). *En un Transformer pre-norm,*

$$H^L = H^0 + \sum_{k=1}^{2L} F_k\big(R^{k-1}\big).$$

Demostración. De la definición 2.1, $R^k - R^{k-1} = F_k(R^{k-1})$. Sumando para $k = 1, \ldots, 2L$, los términos intermedios se cancelan: $R^{2L} - R^0 = \sum_k F_k(R^{k-1})$. Basta usar $R^{2L} = H^L$ y $R^0 = H^0$. ∎

La proposición 2.2 sustenta la lectura del flujo como un *canal de comunicación* [4]: ninguna subcapa sobrescribe el flujo; cada una lee de él (tras normalizar) y le *suma* su salida. De ahí se siguen tres consecuencias.

(i) Una subcapa puede escribir en una dirección de $\mathbb{R}^d$ que lee otra subcapa muchas capas después. Esta descomposición es la base de la interpretabilidad mecanicista (módulo 23).

(ii) Toda subcapa debe producir vectores de $\mathbb{R}^d$, pues su salida se suma al flujo. Esa es la función de W_O en la atención y de W_{d} en el MLP.

(iii) Si una subcapa aprende a escribir casi cero, el modelo se comporta como uno con una subcapa menos.

Ejemplo 2.3 (Lectura de aportaciones). Sea $d = 2$ y $h^0 = (1, 0)$ el vector de un token. Si la atención del primer bloque escribe $(0;\ 0{,}5)$ y el MLP escribe $(-0{,}2;\ 0{,}3)$, entonces $a^1 = (1;\ 0{,}5)$ y $h^1 = (0{,}8;\ 0{,}8)$. Cada aportación se recupera restando vectores consecutivos del flujo, y su componente en una dirección unitaria u es el producto escalar con u.

Proposición 2.4 (Jacobiano del flujo). *Sea J_k el jacobiano de F_k evaluado en R^{k-1}, con todas las matrices vectorizadas. Entonces*

$$\frac{\partial H^L}{\partial H^0} = \prod_{k=2L}^{1} \big(I + J_k\big) = I + \sum_{k=1}^{2L} J_k + \sum_{k>k'} J_k J_{k'} + \cdots + J_{2L} \cdots J_1,$$

donde el producto se ordena de izquierda a derecha con k decreciente.

Demostración. Por la regla de la cadena, $\partial R^k / \partial R^{k-1} = I + J_k$ y $\partial H^L / \partial H^0 = \prod_k \partial R^k / \partial R^{k-1}$ en el orden indicado. El desarrollo se obtiene multiplicando los $2L$ binomios, eligiendo en cada factor I o J_k; los productos se mantienen en orden decreciente de k porque las matrices no conmutan. ■

El término I del desarrollo es el *camino identidad*: el gradiente de la pérdida respecto de H^L llega a H^0 sumado, sin atravesar ninguna normalización ni producto de jacobianos. Es la versión para el Transformer del teorema del jacobiano residual del módulo 11.

2.2 Pre-norm frente a post-norm

Definición 2.5 (Post-norm). El *bloque post-norm* del Transformer original [17, §3.1] aplica la normalización después de la suma residual:

$$A = \mathrm{Norm}_1\big(H + \mathrm{MHA}(H)\big), \qquad H' = \mathrm{Norm}_2\big(A + \mathrm{MLP}(A)\big).$$

Proposición 2.6 (No hay camino identidad en post-norm). *En un Transformer post-norm con normalización RMSNorm sin ganancia,* $\mathrm{Norm}(x) = x/\,\mathrm{RMS}(x)$*, se cumple* $\mathrm{RMS}(h_t^\ell) = 1$ *para toda fila t y toda capa $\ell \geq 1$, y el jacobiano de la salida respecto de la entrada es*

$$\frac{\partial H^L}{\partial H^0} = \prod_{k=2L}^{1} N_k \,(I + J_k),$$

donde N_k es el jacobiano de la normalización k. Cada N_k es singular.

Demostración. La salida de cada bloque es la salida de una normalización, luego tiene RMS igual a 1. La regla de la cadena aplicada a $R^k = \mathrm{Norm}(R^{k-1} + F_k(R^{k-1}))$ da el factor $N_k(I + J_k)$. Para una fila $x \in \mathbb{R}^d$, el jacobiano de $x \mapsto x/\,\mathrm{RMS}(x)$ es $\frac{1}{\mathrm{RMS}(x)}\big(I - \frac{xx^\top}{d\ \mathrm{RMS}(x)^2}\big)$ (módulo 11, tema 3), que anula el vector x: en efecto, $\big(I - \frac{xx^\top}{\|x\|^2}\big)x = 0$ porque $d\ \mathrm{RMS}(x)^2 = \|x\|^2$. Luego cada N_k es singular. ■

En post-norm, toda la señal, y todo el gradiente, atraviesa $2L$ normalizaciones; no existe un término I aislado como en la proposición 2.4. Xiong *et al.* [19, §3.3, teorema 1] cuantifican la diferencia en la inicialización: bajo hipótesis de concentración de las normas, la norma del gradiente de los pesos de la última capa es del orden de $d\sqrt{\ln d}$ en post-norm, sin dependencia de L, y del orden de $d\sqrt{\ln d / L}$ en pre-norm. Gradientes grandes en las capas altas obligan a un calentamiento (*warmup*) largo de la tasa de aprendizaje en post-norm; pre-norm se entrena de forma estable con un calentamiento corto o nulo.

Cuadro 2.1: Comparación de las dos posiciones de la normalización.

Aspecto	Post-norm (2017)	Pre-norm (GPT-2, LLaMA)
Ecuación	$\mathrm{Norm}(x + F(x))$	$x + F(\mathrm{Norm}(x))$
Camino identidad	no (proposición 2.6)	sí (proposición 2.4)
Norma del flujo	fija	crece con la profundidad
Calentamiento	imprescindible	tolerante
Normalización final	innecesaria	necesaria

2.3 El precio de pre-norm: crecimiento del flujo

Proposición 2.7 (Crecimiento del flujo en un Transformer). *Supóngase que, en la inicialización, las aportaciones $F_k(R^{k-1})_t$ de una fila son vectores aleatorios de media nula, incorrelados entre sí y con h_t^0, con $\mathbb{E}\|F_k(R^{k-1})_t\|^2 = s^2$. Entonces*

$$\mathbb{E}\|h_t^L\|^2 = \mathbb{E}\|h_t^0\|^2 + 2L\, s^2.$$

Demostración. Por la proposición 2.2, $h_t^L = h_t^0 + \sum_{k=1}^{2L} F_k(R^{k-1})_t$. Al desarrollar $\|h_t^L\|^2$ como producto escalar de la suma consigo misma, los términos cruzados tienen esperanza nula por la incorrelación y la media nula; quedan las $1 + 2L$ normas al cuadrado. ∎

La proposición 2.7 tiene dos consecuencias prácticas. Primera: como la norma del flujo crece, el vector h_t^L debe normalizarse antes del unembedding; de lo contrario, la escala de los logits dependería de la profundidad. Es la normalización final Norm_f de la definición 1.3, que GPT-2 introdujo junto con la posición pre-norm [12, §2.3] y que Jurafsky y Martin señalan como la capa adicional que exige pre-norm [8, §7.2.3]. Segunda: GPT-2 multiplica en la inicialización los pesos de las capas que escriben en el flujo (W_O y la última matriz del MLP) por $1/\sqrt{N}$, con $N = 2L$ el número de capas residuales; así cada s^2 se divide por $2L$ y la norma esperada del flujo final queda acotada independientemente de la profundidad.

Ejemplo 2.8. En GPT-2 *small*, $L = 12$ y hay $N = 24$ capas residuales. Si $\mathbb{E}\|h_t^0\|^2 = 1$ y cada subcapa aporta $s^2 = 1$, la proposición 2.7 da $\mathbb{E}\|h_t^{12}\|^2 = 25$. Con el factor $1/\sqrt{24} \approx 0{,}2041$ en los pesos de salida, $s^2 = 1/24$ y $\mathbb{E}\|h_t^{12}\|^2 = 1 + 24/24 = 2$.

Observación 2.9 (Conexión con los LLM). Casi todos los modelos de lenguaje abiertos posteriores a GPT-2 usan pre-norm; la práctica de CS336 la describe como una de las pocas modificaciones de consenso respecto del Transformer original [6, §7.3]. LLaMA sustituye LayerNorm por RMSNorm [21], que no centra el vector ni tiene sesgo, y normaliza la entrada de cada subcapa para mejorar la estabilidad del entrenamiento [15, §2.2].

Lecturas recomendadas para este tema

- Jurafsky y Martin [8], §7.2 «Transformer Blocks» y §7.2.2 «Layer Norm».
- Xiong *et al.* [19], §3.3 «Understanding the Transformer at initialization».
- Radford *et al.* [12], §2.3 «Model».
- Elhage *et al.* [4], apartado sobre el flujo residual como canal de comunicación.
- Hashimoto y Liang [6], §3.4 «Pre-Norm Transformer Block».

Ejercicios del tema 2

Ejercicio 2.1 ⋆ Escriba las ecuaciones de un bloque pre-norm y de uno post-norm. Sustituya la primera ecuación en la segunda en cada caso y determine en cuál existe un camino desde la entrada x hasta la salida que no atraviesa ninguna normalización.

Fuente: adaptado de Hashimoto y Liang [6], §3.4 y §7.3, ecuaciones (25) a (28) y problema `pre_norm_ablation`; el experimento de entrenamiento se sustituye por el análisis de las ecuaciones.

Ejercicio 2.2 ⋆ Explique por qué un Transformer pre-norm necesita una normalización final antes del unembedding y uno post-norm no.

Ejercicio 2.3 ⋆⋆ Con los datos del ejemplo 2.3 y RMSNorm sin ganancia, $\mathrm{Norm}(x) = x/\,\mathrm{RMS}(x)$ con $\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_i x_i^2}$, calcule: (a) la entrada normalizada de la atención, $\mathrm{Norm}(h^0)$; (b) la entrada normalizada del MLP, $\mathrm{Norm}(a^1)$; (c) $\mathrm{Norm}(h^1)$; (d) la componente de cada aportación en la dirección $u = (0, 1)$.

Ejercicio 2.4 ⋆⋆ Un modelo pre-norm tiene $L = 32$ bloques. En la inicialización, $\mathbb{E}\|h_t^0\|^2 = 1$ y cada subcapa aporta $s^2 = 0{,}5$. (a) Calcule $\mathbb{E}\|h_t^L\|^2$ y el cociente entre la norma cuadrática media final y la inicial. (b) ¿Por qué factor deberían escalarse los pesos de salida de las subcapas para que $\mathbb{E}\|h_t^L\|^2 = 1{,}5$?

Ejercicio 2.5 ⋆⋆⋆ Supóngase que los jacobianos de la proposición 2.4 cumplen $\|J_k\|_2 \le \varepsilon$ para todo k. Demuestre que

$$\left\| \frac{\partial H^L}{\partial H^0} - I \right\|_2 \le (1+\varepsilon)^{2L} - 1,$$

y deduzca que, si $2L\varepsilon \le 1$, la desviación respecto de la identidad es a lo sumo $(e-1)\,2L\varepsilon$. Interprete el resultado.

Tema 3

La subcapa MLP: GELU y SwiGLU

La segunda subcapa de cada bloque es un perceptrón de una capa oculta que se aplica por separado a cada token. Contiene unos dos tercios de los parámetros del modelo. Este tema define el MLP clásico del Transformer original y de GPT-2, las unidades lineales con puerta y la variante SwiGLU de LLaMA, demuestra las propiedades de la activación SiLU y justifica la elección $d_{\text{ff}} \approx \frac{8}{3}d$. Se sigue [17, §3.3], [20, §11.7.2], [8, §7.2.1], el artículo de Shazeer [13] y la §3.4.2 de la práctica de CS336 [6].

3.1 El MLP posición a posición

Definición 3.1 (MLP posición a posición). Sea $\phi : \mathbb{R} \to \mathbb{R}$ una función de activación aplicada componente a componente. El *MLP posición a posición* con dimensión intermedia d_{ff} es

$$\text{FFN}(x) = \phi\big(xW_1 + b_1\big)W_2 + b_2, \qquad W_1 \in \mathbb{R}^{d\times d_{\text{ff}}},\ W_2 \in \mathbb{R}^{d_{\text{ff}}\times d},$$

aplicado a cada fila $x \in \mathbb{R}^d$ de la matriz de entrada con los mismos parámetros.

El Transformer original usa $\phi = \text{ReLU}$, $d = 512$ y $d_{\text{ff}} = 2048 = 4d$ [17, §3.3]; GPT-2 usa la misma proporción con la activación GELU. Que el MLP actúe fila a fila tiene una consecuencia estructural.

Lema 3.2 (Las operaciones fila a fila conmutan con las permutaciones). *Sea $f : \mathbb{R}^d \to \mathbb{R}^m$ y sea $\hat{f} : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times m}$ la aplicación que actúa como f sobre cada fila. Para toda matriz de permutación $P \in \mathbb{R}^{n\times n}$, $\hat{f}(PX) = P\,\hat{f}(X)$.*

Demostración. La fila t de PX es la fila $\pi(t)$ de X, para cierta permutación π. La fila t de $\hat{f}(PX)$ es, por tanto, $f(x_{\pi(t)})$, que es la fila $\pi(t)$ de $\hat{f}(X)$, es decir, la fila t de $P\hat{f}(X)$. ∎

El lema 3.2 se aplica al MLP, a las normalizaciones y a las proyecciones lineales. Por eso la atención es la única subcapa sensible al orden de los tokens, y por eso hace falta codificar la posición (tema 4).

3.2 Activaciones suaves: GELU y SiLU

Definición 3.3 (GELU y SiLU). Sean Φ la función de distribución de la normal estándar y $\sigma(x) = 1/(1 + e^{-x})$ la sigmoide. Se definen

$$\text{GELU}(x) = x\,\Phi(x), \qquad \text{SiLU}(x) = x\,\sigma(x).$$

La función SiLU también se llama *Swish* con parámetro 1 [13].

Ambas multiplican x por un número entre 0 y 1 que crece con x: son versiones suaves de $\mathrm{ReLU}(x) = x \cdot \mathbf{1}[x > 0]$, en las que el interruptor $\mathbf{1}[x > 0]$ se sustituye por una función de distribución [7].

Proposición 3.4 (Propiedades de SiLU). *La función SiLU cumple:*

(i) $\mathrm{SiLU}(x) - \mathrm{ReLU}(x) \to 0$ *cuando* $x \to \pm\infty$*;*

(ii) $\mathrm{SiLU}'(x) = \sigma(x)\big(1 + x(1 - \sigma(x))\big)$*; en particular,* $\mathrm{SiLU}(0) = 0$ *y* $\mathrm{SiLU}'(0) = 1/2$*;*

(iii) tiene un único punto crítico $x^* < 0$*, que es un mínimo global, con* $\mathrm{SiLU}(x^*) = x^* + 1$*. Numéricamente,* $x^* \approx -1{,}2785$ *y* $\mathrm{SiLU}(x^*) \approx -0{,}2785$*.*

Demostración. (i) Para $x > 0$, $\mathrm{SiLU}(x) - \mathrm{ReLU}(x) = x(\sigma(x) - 1) = -x/(1 + e^x) \to 0$. Para $x < 0$, $\mathrm{SiLU}(x) - 0 = x/(1 + e^{-x}) = xe^x/(1 + e^x) \to 0$, porque $xe^x \to 0$ cuando $x \to -\infty$.

(ii) Con $\sigma' = \sigma(1-\sigma)$ y la regla del producto, $\mathrm{SiLU}'(x) = \sigma(x) + x\sigma(x)(1-\sigma(x))$. En $x = 0$, $\sigma(0) = 1/2$.

(iii) Como $\sigma(x) > 0$, los puntos críticos son las raíces de $g(x) = 1 + x(1 - \sigma(x))$. Para $x \geq 0$, $g(x) \geq 1 > 0$. En $(-\infty, 0)$,

$$g'(x) = (1 - \sigma) - x\sigma(1 - \sigma) = (1 - \sigma(x))\big(1 - x\sigma(x)\big) > 0,$$

luego g es estrictamente creciente; además $g(0) = 1$ y $g(x) \to -\infty$ cuando $x \to -\infty$, porque $1 - \sigma(x) \to 1$. Por el teorema de Bolzano y la monotonía, g tiene una única raíz $x^* < 0$, con $g < 0$ a su izquierda y $g > 0$ a su derecha: SiLU decrece y luego crece, y x^* es el mínimo global. De $g(x^*) = 0$ se obtiene $\sigma(x^*) = 1 + 1/x^*$, luego $\mathrm{SiLU}(x^*) = x^*\sigma(x^*) = x^* + 1$. La raíz numérica se obtiene por bisección. ∎

A diferencia de ReLU, SiLU es derivable en todo punto, no es monótona y deja pasar valores negativos pequeños, lo que evita las neuronas cuyo gradiente es nulo en todo el semieje negativo.

3.3 Unidades lineales con puerta y SwiGLU

Definición 3.5 (SwiGLU). Sean $W_g, W_u \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$ y $W_{\mathrm{d}} \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$. La subcapa *SwiGLU* es

$$\mathrm{SwiGLU}(x) = \big(\mathrm{SiLU}(xW_g) \odot xW_u\big)\, W_{\mathrm{d}},$$

donde $\odot$ es el producto componente a componente. La rama xW_g es la *puerta* y xW_u la *rama lineal*.

Las unidades lineales con puerta (GLU) de Dauphin *et al.* [3] multiplican una proyección lineal por una puerta sigmoide; Shazeer [13, §2] probó varias funciones en la puerta y encontró que la variante con Swish mejoraba la perplejidad del Transformer. Las componentes de la capa intermedia ya no son ϕ(lineal), sino productos de dos formas lineales del token: la puerta decide cuánto de la rama lineal se transmite.

Proposición 3.6 (Presupuesto de parámetros). *Sin sesgos, el MLP de la definición 3.1 tiene* $2d\, d_{\mathrm{ff}}$ *parámetros, y SwiGLU tiene* $3d\, d_{\mathrm{ff}}$*. Con* $d_{\mathrm{ff}} = 4d$*, el primero tiene* $8d^2$ *parámetros; SwiGLU tiene el mismo número si y solo si* $d_{\mathrm{ff}} = \frac{8}{3}d$*.*

Demostración. W_1 y W_2 tienen $d\,d_{\text{ff}}$ entradas cada una; W_g, W_u y W_{d}, también. Con $d_{\text{ff}} = 4d$: $2d \cdot 4d = 8d^2$. La ecuación $3d\,d_{\text{ff}} = 8d^2$ tiene la única solución $d_{\text{ff}} = 8d/3$. ■

El mismo argumento iguala el cómputo, porque el coste de cada producto es proporcional al número de pesos (tema 6). Shazeer [13, §2] reduce d_{ff} en un factor 2/3 por esta razón: en sus experimentos, el MLP de dos matrices usa $d_{\text{ff}} = 3072$ y las variantes con puerta $d_{\text{ff}} = 2048$.

Ejemplo 3.7 (LLaMA). El código publicado de LLaMA [10] calcula la dimensión intermedia como $\lfloor \frac{2}{3} \cdot 4d \rfloor$ y la redondea hacia arriba al múltiplo de 256 más próximo. Para $d = 4096$ (LLaMA-2 7B): $\lfloor 10\,922{,}67 \rfloor = 10\,922$ y $\lceil 10\,922/256 \rceil = 43$, luego $d_{\text{ff}} = 43 \cdot 256 = 11\,008$. El MLP tiene $3 \cdot 4096 \cdot 11\,008 = 135\,266\,304$ parámetros, un 0,78 % más que $8d^2 = 134\,217\,728$. Para $d = 5120$ (LLaMA-2 13B), el mismo cálculo da $d_{\text{ff}} = 13\,824$. El redondeo a múltiplos de potencias de 2 aprovecha mejor las unidades de multiplicación de matrices de las GPU (módulo 17).

Observación 3.8 (Conexión con los LLM). SwiGLU es el MLP de LLaMA [15, §2.2], de Llama 3 y de Qwen 2.5, según la práctica de CS336 [6, §3.4.2]. Shazeer no ofrece una explicación teórica de la mejora; la práctica propone comprobarla empíricamente comparando SwiGLU con un MLP SiLU sin puerta de $d_{\text{ff}} = 4d$, que tiene aproximadamente el mismo número de parámetros (problema `swiglu_ablation`). En los modelos con mezcla de expertos (módulo 18), cada experto es una subcapa de este tipo.

Lecturas recomendadas para este tema

- Vaswani *et al.* [17], §3.3 «Position-wise Feed-Forward Networks».
- Zhang *et al.* [20], §11.7.2 «Positionwise Feed-Forward Networks».
- Jurafsky y Martin [8], §7.2.1 «Feedforward layer».
- Shazeer [13], §2 «Gated Linear Units (GLU) and Variants».
- Hashimoto y Liang [6], §3.4.2 «Position-Wise Feed-Forward Network».

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Explique por qué el MLP no mezcla información entre tokens. Cuente los parámetros del MLP del Transformer base de Vaswani *et al.* ($d = 512$, $d_{\text{ff}} = 2048$, con sesgos) y diga qué fracción corresponde a los sesgos.

Ejercicio 3.2 ⋆⋆ Justifique $d_{\text{ff}} = \frac{8}{3}d$ en SwiGLU. Para $d = 4096$, calcule $\frac{8}{3}d$, el múltiplo de 256 inmediatamente superior y cuántos parámetros de más tiene el MLP de LLaMA-2 7B respecto de $8d^2$.

Ejercicio 3.3 ⋆⋆ Sea $d = d_{\text{ff}} = 2$, $x = (1, -1)$ y

$$W_g = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}, \qquad W_u = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}, \qquad W_{\text{d}} = \begin{pmatrix} 1 & -1 \\ 2 & 0 \end{pmatrix}.$$

Calcule $\text{SwiGLU}(x)$ indicando la puerta, la rama lineal y la capa intermedia. Use $\sigma(1) = 0{,}7311$.

Ejercicio 3.4 ⋆⋆ Para un modelo con $d = 1600$: (a) compruebe que $d_{\text{ff}} = 4288$ es el múltiplo de 64 más cercano a $\frac{8}{3}d$; (b) cuente los parámetros de la subcapa SwiGLU con ese d_{ff} y los de un MLP SiLU sin puerta, $\text{FFN}_{\text{SiLU}}(x) = \text{SiLU}(xW_1)W_2$, con $d_{\text{ff}} = 4d$, ambos sin sesgos; (c) calcule el cociente entre ambos. (d) Repita (a) y (b) para $d = 768$.

Fuente: adaptado de Hashimoto y Liang [6], §3.4.2, problema `positionwise_feedforward`, y §7.3, problema `swiglu_ablation` (se sustituye el experimento por el recuento).

Ejercicio 3.5 ⋆⋆⋆ Demuestre las propiedades (ii) y (iii) de la proposición 3.4 sin consultar la demostración. Después demuestre que $\text{SiLU}(x) = \frac{x}{2}\big(1 + \tanh\frac{x}{2}\big)$ y deduzca que $\text{SiLU}(x) - \text{SiLU}(-x) = x$ y que $\text{SiLU}(x) + \text{SiLU}(-x) = x\tanh\frac{x}{2}$. ¿Qué dice la primera identidad sobre la parte impar de SiLU?

Tema 4

Codificación de la posición: sinusoidal y RoPE

Sin información de posición, la atención trata la secuencia como un conjunto. Este tema demuestra esa propiedad, define la codificación sinusoidal del Transformer original y las posiciones aprendidas de GPT-2, y estudia con detalle la codificación rotatoria RoPE de LLaMA, cuya puntuación de atención solo depende de la distancia entre tokens. Se sigue [17, §3.5], [20, §11.6.3], [1, §12.1.9], [8, §7.4], el artículo de Su *et al.* [14, §3.2] y las secciones 2 y 3 de la práctica 4 de CS224N [2].

4.1 La atención no ve el orden

Proposición 4.1 (Equivariancia por permutaciones). *Sea*

$$f(X) = \mathrm{softmax}\big(XW_Q(XW_K)^\top/\sqrt{d_k}\big)XW_V$$

la autoatención sin máscara*, con la softmax por filas. Para toda matriz de permutación P, $f(PX) = P\,f(X)$.*

Demostración. Sea $S = XW_QW_K^\top X^\top/\sqrt{d_k}$. Con PX en lugar de X, la matriz de puntuaciones es $PSP^\top$, cuya entrada (t,u) es $S_{\pi(t)\pi(u)}$. La softmax por filas de $PSP^\top$ es $P\,\mathrm{softmax}(S)P^\top$: la fila t contiene los mismos números que la fila $\pi(t)$ de S, reordenados por π, y la softmax de un vector reordenado es la softmax reordenada. Por tanto $f(PX) = P\,\mathrm{softmax}(S)P^\top PXW_V = P\,\mathrm{softmax}(S)XW_V = Pf(X)$, porque $P^\top P = I$. ∎

Junto con el lema 3.2, la proposición 4.1 implica que un Transformer sin máscara y sin posiciones es equivariante: barajar los tokens de la entrada solo baraja las filas de la salida, de modo que «el perro muerde al hombre» y «el hombre muerde al perro» producen los mismos vectores por palabra. La máscara causal rompe la simetría (ejercicio 1.6), y la práctica de CS336 recuerda que un decoder causal puede, en principio, inferir posiciones sin codificación explícita [6, §7.3]; aun así, todos los modelos de la cuadro 1.1 codifican la posición de forma explícita.

4.2 Codificación sinusoidal y posiciones aprendidas

Definición 4.2 (Codificación sinusoidal). Sea d par y $\omega_i = 10000^{-2i/d}$ para $i = 0,\dots,d/2-1$. La *codificación sinusoidal* de la posición $p \in \mathbb{N}$ es el vector $\mathrm{PE}(p) \in \mathbb{R}^d$ con componentes

$$\mathrm{PE}(p)_{2i} = \sin(p\,\omega_i), \qquad \mathrm{PE}(p)_{2i+1} = \cos(p\,\omega_i),$$

que se suma al embedding del token: $h_t^0 = e_{x_t} + \mathrm{PE}(t)$ [17, §3.5].

Cada par $(2i, 2i+1)$ es un punto que gira sobre la circunferencia unidad con frecuencia angular ω_i; las frecuencias decrecen geométricamente desde $\omega_0 = 1$ hasta $\omega_{d/2-1} \approx 10^{-4}$. Es como un reloj con $d/2$ agujas que giran a velocidades distintas [20, §11.6.3.1].

Ejemplo 4.3. Con $d = 4$: $\omega_0 = 1$ y $\omega_1 = 10000^{-1/2} = 0{,}01$. Entonces $\mathrm{PE}(1) = (0{,}8415;\ 0{,}5403;\ 0{,}0100;\ 1{,}0000)$ y $\mathrm{PE}(2) = (0{,}9093;\ -0{,}4161;\ 0{,}0200;\ 0{,}9998)$, redondeando a cuatro decimales. La norma es constante: $\|\mathrm{PE}(p)\|^2 = d/2 = 2$ para todo p.

Teorema 4.4 (Desplazamiento lineal). *Para cada $k \in \mathbb{Z}$ existe una matriz ortogonal $M_k \in \mathbb{R}^{d\times d}$, que no depende de p, tal que $\mathrm{PE}(p+k) = M_k\,\mathrm{PE}(p)$ para todo p.*

Demostración. Se fija un par i y se escribe $a = p\,\omega_i$, $b = k\,\omega_i$. Por las fórmulas del seno y del coseno de la suma,

$$\begin{pmatrix}\sin(a+b)\\ \cos(a+b)\end{pmatrix} = \begin{pmatrix}\cos b & \sin b\\ -\sin b & \cos b\end{pmatrix}\begin{pmatrix}\sin a\\ \cos a\end{pmatrix}.$$

La matriz 2×2 solo depende de $b = k\omega_i$ y es una rotación, luego ortogonal. Sea M_k la matriz diagonal por bloques con estos $d/2$ bloques; es ortogonal porque lo es cada bloque, y la igualdad se cumple par a par. ∎

El teorema 4.4 es la afirmación de Vaswani *et al.* [17, §3.5] de que $\mathrm{PE}(p+k)$ es una función lineal de $\mathrm{PE}(p)$ para cada desplazamiento fijo, y la propiedad de posición relativa de [20, §11.6.3.2]. La esperanza era que la red aprendiera a atender por posiciones relativas mediante esa aplicación lineal.

Definición 4.5 (Posiciones aprendidas). Un modelo con *posiciones aprendidas* añade una matriz de parámetros $W_P \in \mathbb{R}^{n_{\mathrm{ctx}}\times d}$ y suma su fila t al embedding del token t. Es la opción de GPT-2 [12]. Aporta $n_{\mathrm{ctx}}d$ parámetros y no define ninguna codificación para posiciones $t > n_{\mathrm{ctx}}$.

4.3 Codificación rotatoria (RoPE)

En lugar de sumar la posición al embedding, RoPE la introduce en cada capa, girando las consultas y las claves [14]. Sea

$$R(\alpha) = \begin{pmatrix}\cos\alpha & -\sin\alpha\\ \sin\alpha & \cos\alpha\end{pmatrix}$$

la rotación plana de ángulo α.

Lema 4.6 (Rotaciones planas). *Para todos $a, b \in \mathbb{R}$: (i) $R(a)R(b) = R(a+b)$; (ii) $R(a)^\top = R(-a)$; (iii) en consecuencia, $R(a)^\top R(b) = R(b-a)$ y $R(a)$ es ortogonal.*

Demostración. (i) La entrada $(1,1)$ del producto es $\cos a\cos b - \sin a\sin b = \cos(a+b)$ y la entrada $(2,1)$ es $\sin a\cos b + \cos a\sin b = \sin(a+b)$; las entradas $(1,2)$ y $(2,2)$ son $-\sin(a+b)$ y $\cos(a+b)$ por el mismo cálculo. (ii) Al trasponer se intercambian los senos de fuera de la diagonal; como $\cos(-a) = \cos a$ y $\sin(-a) = -\sin a$, el resultado es $R(-a)$. (iii) $R(a)^\top R(b) = R(-a)R(b) = R(b-a)$ por (ii) y (i); con $b = a$ se obtiene $R(a)^\top R(a) = R(0) = I$. ∎

Definición 4.7 (RoPE). Sea d_k par y $\theta_i = 10000^{-2i/d_k}$ para $i = 0, \ldots, d_k/2 - 1$. Para cada posición m se define la matriz diagonal por bloques

$$R_m = \mathrm{diag}\big(R(m\theta_0),\, R(m\theta_1),\, \ldots,\, R(m\theta_{d_k/2-1})\big) \in \mathbb{R}^{d_k\times d_k}.$$

RoPE sustituye, en cada cabeza y cada capa, la consulta q_m del token m por $\tilde{q}_m = R_m q_m$ y la clave k_n del token n por $\tilde{k}_n = R_n k_n$. Los valores v_n no se modifican.

Teorema 4.8 (RoPE es relativa). *Para todos $q, k \in \mathbb{R}^{d_k}$ y todas las posiciones m, n,*

$$(R_m q)^\top (R_n k) = q^\top R_{n-m}\, k.$$

En particular, la puntuación de atención entre el token m y el token n solo depende de las posiciones a través de la diferencia $n - m$.

Demostración. La traspuesta de una matriz diagonal por bloques es la matriz diagonal por bloques de las traspuestas, y el producto de dos matrices diagonales por bloques con bloques del mismo tamaño se calcula bloque a bloque. Por el lema 4.6(iii) aplicado a cada bloque,

$$R_m^\top R_n = \operatorname{diag}\big(R(m\theta_i)^\top R(n\theta_i)\big)_i = \operatorname{diag}\big(R((n-m)\theta_i)\big)_i = R_{n-m}.$$

Finalmente, $(R_m q)^\top (R_n k) = q^\top R_m^\top R_n k = q^\top R_{n-m} k$. ∎

Corolario 4.9. *RoPE conserva las normas: $\|R_m q\| = \|q\|$. Solo modifica el ángulo entre la consulta y la clave.*

Demostración. R_m es ortogonal por el lema 4.6, luego $\|R_m q\|^2 = q^\top R_m^\top R_m q = q^\top q$. ∎

Para un solo par ($d_k = 2$), si $\varphi = (n - m)\theta$, el desarrollo de $q^\top R(\varphi) k$ es

$$q^\top R(\varphi)\, k = (q \cdot k)\cos\varphi - (q_1 k_2 - q_2 k_1)\sin\varphi. \tag{4.1}$$

Ejemplo 4.10 (Misma distancia, misma puntuación). Sean $d_k = 2$, $\theta = \pi/6$, $q = (1, 2)$ y $k = (3, 1)$, de modo que $q \cdot k = 5$ y $q_1 k_2 - q_2 k_1 = -5$. Por (4.1), para $n - m = -2$ la puntuación es $5\cos(-\pi/3) + 5\sin(-\pi/3) = 2{,}5 - 4{,}330 = -1{,}830$, y para $n - m = +2$ es $2{,}5 + 4{,}330 = 6{,}830$. La cuadro 4.1 muestra que tres pares de posiciones con la misma diferencia dan la misma puntuación.

Cuadro 4.1: Puntuaciones $(R_m q)^\top (R_n k)$ del ejemplo 4.10.

m (consulta)	n (clave)	$n - m$	Puntuación
3	1	-2	$-1{,}830$
5	3	-2	$-1{,}830$
2	0	-2	$-1{,}830$
1	3	$+2$	6,830

Proposición 4.11 (Forma compleja). *Identifíquese cada par (a, b) con el número complejo $z = a + ib$. Entonces la rotación $R(\alpha)$ equivale a multiplicar por $e^{i\alpha}$, y, con $\langle u, v\rangle = \operatorname{Re}(u\bar{v})$, el producto escalar de dos pares rotados es*

$$\big\langle z_1 e^{im\theta},\, z_2 e^{in\theta} \big\rangle = \operatorname{Re}\big(z_1 \bar{z}_2\, e^{i(m-n)\theta}\big).$$

Demostración. $e^{i\alpha}(a+ib) = (a\cos\alpha - b\sin\alpha) + i(a\sin\alpha + b\cos\alpha)$, que es el vector $R(\alpha)(a, b)^\top$. Para $u = a + ib$ y $v = c + id$, $\operatorname{Re}(u\bar{v}) = ac + bd$ es el producto escalar real. Finalmente, $\overline{z_2 e^{in\theta}} = \bar{z}_2 e^{-in\theta}$, y los exponentes se suman. ∎

La forma compleja es la que usan el artículo original [14, §3.2.1] y el código de LLaMA [10], que agrupa las componentes consecutivas de q y k en números complejos. La fase $e^{i(m-n)\theta}$ hace el papel del desfase entre dos osciladores de la misma frecuencia: solo importa la diferencia de fases, como en la interferencia de dos ondas.

Cuadro 4.2: Comparación de las codificaciones de posición. La propiedad relativa es la del teorema 4.4 para la sinusoidal y la del teorema 4.8 para RoPE.

Método	Dónde actúa	Parámetros	Relativa	Modelos
Sinusoidal	suma al embedding	0	indirecta	Transformer
Aprendida	suma al embedding	$n_{\text{ctx}}d$	no	GPT-2, GPT-3
RoPE	gira q y k en cada capa	0	exacta	LLaMA, Qwen

Observación 4.12 (Conexión con los LLM). RoPE no gira los valores v: la posición debe influir en *a quién* se atiende, no en *qué* contenido se copia al flujo residual. La práctica de CS336 precalcula los cosenos y senos una sola vez y los comparte entre capas, porque RoPE no tiene parámetros entrenables [6, §3.4.3]. Los métodos para ampliar el contexto de un modelo con RoPE (módulo 16) modifican las frecuencias θ_i; la más lenta tiene periodo $2\pi \cdot 10000^{(d_k-2)/d_k}$, que con la dimensión por cabeza de LLaMA-2 7B, $d_k = 128$, vale unas 54 410 posiciones.

Lecturas recomendadas para este tema

- Vaswani *et al.* [17], §3.5 «Positional Encoding».
- Zhang *et al.* [20], §11.6.3 «Positional Encoding» (posición absoluta y relativa).
- Bishop y Bishop [1], §12.1.9 «Positional encoding».
- Su *et al.* [14], §3.2 «Rotary position embedding».
- CS224N [2], práctica 4, sección 2 «Position Embeddings Exploration».

Ejercicios del tema 4

Ejercicio 4.1 ⋆ ¿Por qué RoPE gira q y k pero no v? ¿Qué ocurriría si también girase v?

Ejercicio 4.2 ⋆⋆ Calcule PE(1) y PE(2) con $d = 4$ y compruebe que cada par cumple $\sin^2 + \cos^2 = 1$.

Ejercicio 4.3 ⋆⋆ Con $d = 2$ ($\omega_0 = 1$), escriba la matriz M_1 del teorema 4.4 con cuatro decimales y compruebe numéricamente que $M_1\,\text{PE}(1) = \text{PE}(2)$.

Ejercicio 4.4 ⋆⋆ Con $d_k = 2$, $\theta = \pi/6$, $q = (1, 2)$ y $k = (3, 1)$, calcule $R(3\theta)q$, $R(\theta)k$ y su producto escalar. Compruebe que coincide con $q^\top R(-2\theta)k$ y que $\|R(\theta)k\| = \|k\|$.

Ejercicio 4.5 ⋆⋆⋆ Demuestre el teorema 4.8 para RoPE en d_k dimensiones, justificando cada propiedad de las matrices diagonales por bloques que se use. Compruebe el caso $m = n$.

Ejercicio 4.6 ⋆⋆⋆ Una capa calcula $Q = XW_Q$, $K = XW_K$, $V = XW_V$, $H = \text{softmax}(QK^\top/\sqrt{d})V$ y $Z = \text{ReLU}(HW_1 + \mathbf{1}b_1)W_2 + \mathbf{1}b_2$, con $X \in \mathbb{R}^{T\times d}$ y $\mathbf{1}$ un vector

columna de unos. (a) Demuestre que, si $X' = PX$ con P de permutación, entonces $Z' = PZ$; puede usar $\mathrm{softmax}(PAP^\top) = P\,\mathrm{softmax}(A)P^\top$ y $\mathrm{ReLU}(PA) = P\,\mathrm{ReLU}(A)$. (b) Explique por qué es un problema para modelar texto. (c) ¿Pueden dos posiciones enteras distintas recibir la misma codificación sinusoidal?

Fuente: adaptado de CS224N [2], práctica 4, sección 2, apartados (a) y (b).

Ejercicio 4.7 ⋆ ⋆ ⋆ Escriba cada par de componentes como un complejo y defina $\mathrm{RoPE}(z, t) = z\,e^{it\theta}$. (a) Demuestre que $\langle \mathrm{RoPE}(z_1, t_1), \mathrm{RoPE}(z_2, t_2)\rangle = \langle \mathrm{RoPE}(z_1, t_1 - t_2), \mathrm{RoPE}(z_2, 0)\rangle$, con $\langle u, v\rangle = \mathrm{Re}(u\bar{v})$. (b) Compruébelo con $z_1 = 1 + 2i$, $z_2 = 3 + i$, $\theta = \pi/6$, $t_1 = 3$ y $t_2 = 1$.

Fuente: adaptado de CS224N [2], práctica 4, sección 3, apartado (g)(ii) «Relative Embeddings».

Tema 5

Contar parámetros y memoria

A partir de la configuración de un Transformer (V, d, L, h, d_{ff}) puede calcularse sin ejecutar nada cuántos parámetros tiene y cuánta memoria necesita para la inferencia y para el entrenamiento. Este tema deduce la fórmula exacta para la arquitectura de LLaMA, la aproximación $N \approx 12Ld^2$ de Kaplan *et al.* [9, §2.1] y el coste en bytes por parámetro del entrenamiento con Adam en precisión mixta. Se sigue también [8, §7.1.1 y ejercicio 7.1] y el problema `transformer_accounting` de CS336 [6].

5.1 Parámetros de un bloque

Proposición 5.1 (La atención no depende del número de cabezas). *Sin sesgos, la atención multicabeza con h cabezas de dimensión $d_k = d/h$ tiene $4d^2$ parámetros, cualquiera que sea h.*

Demostración. Cada cabeza tiene tres matrices $d \times d_k$, con $3dd_k$ parámetros; las h cabezas suman $3hdd_k = 3d^2$. La proyección de salida W_O tiene d^2. En total, $4d^2$. ∎

Proposición 5.2 (Parámetros de un bloque). *(i) Un bloque de tipo LLaMA (RMSNorm con ganancia, SwiGLU, sin sesgos) tiene $4d^2 + 3d\,d_{\text{ff}} + 2d$ parámetros.*

(ii) Un bloque de tipo GPT-2 (LayerNorm con ganancia y sesgo, MLP con $d_{\text{ff}} = 4d$, sesgos en todas las capas lineales) tiene $12d^2 + 13d$ parámetros.

(iii) Con $d_{\text{ff}} = 4d$ en un MLP de dos matrices o $d_{\text{ff}} = \frac{8}{3}d$ en SwiGLU, la parte matricial del bloque tiene $12d^2$ parámetros: $4d^2$ en la atención y $8d^2$ en el MLP.

Demostración. (i) Atención: $4d^2$ (proposición 5.1). SwiGLU: $3d\,d_{\text{ff}}$ (proposición 3.6). Dos RMSNorm con un vector de ganancia de dimensión d cada una: $2d$.

(ii) Atención: $3d^2 + d^2$ en matrices y $3d + d$ en sesgos. MLP: $4d^2 + 4d^2$ en matrices y $4d + d$ en sesgos. Dos LayerNorm: $2(d + d) = 4d$. Matrices: $12d^2$; vectores: $4d + 5d + 4d = 13d$.

(iii) Es la proposición 3.6 sumada a la proposición 5.1. ∎

5.2 Parámetros del modelo completo

Teorema 5.3 (Recuento exacto). *Un Transformer de tipo LLaMA con embeddings sin atar tiene*

$$N = L\big(4d^2 + 3d\,d_{\text{ff}} + 2d\big) + 2Vd + d$$

parámetros. Con embeddings atados, el término $2Vd$ se sustituye por Vd; con posiciones aprendidas, se añaden $n_{\text{ctx}}d$.

Demostración. Hay L bloques (proposición 5.2); la matriz de embedding y la de unembedding tienen Vd parámetros cada una; la normalización final tiene d. Las variantes se siguen de la proposición 1.8 y de la definición de posiciones aprendidas. ■

Corolario 5.4 (Aproximación de Kaplan). *Si $d_{\text{ff}} = 4d$ (o $\frac{8}{3}d$ en SwiGLU), los parámetros sin contar embeddings ni normalizaciones son exactamente $12Ld^2$, y*

$$\frac{N}{12Ld^2} = 1 + \frac{V}{6Ld} + \frac{2L+1}{12Ld} \qquad \textit{(sin atar).}$$

Demostración. Sustituyendo $3d\,d_{\text{ff}} = 8d^2$ en el teorema 5.3, $N = 12Ld^2 + 2Ld + 2Vd + d$. Dividiendo por $12Ld^2$ se obtiene la expresión. ■

Kaplan *et al.* [9, §2.1, ecuación (2.1) y tabla 1] definen N como el número de parámetros *sin embeddings*, $N \approx 2d\,L(2d_{\text{attn}} + d_{\text{ff}}) = 12Ld^2$, porque observaron que las leyes de escalado se expresan mejor con esa cantidad (módulo 16).

Ejemplo 5.5 (LLaMA-2 7B). La configuración publicada es $d = 4096$, $L = 32$, $h = 32$, $d_{\text{ff}} = 11\,008$ y $V = 32\,000$, con embeddings sin atar [15, tabla 2], [10]. La cuadro 5.1 aplica la proposición 5.2. Por el teorema 5.3,

$$N = 32 \cdot 202\,383\,360 + 2 \cdot 131\,072\,000 + 4096 = 6\,738\,415\,616.$$

La aproximación $12Ld^2 = 6\,442\,450\,944$ cubre el 95,6 % de N; los embeddings suponen el 3,9 %. El MLP contiene el 64,2 % de todos los parámetros y la atención el 31,9 %.

Cuadro 5.1: Parámetros de un bloque de LLaMA-2 7B.

Pieza	Fórmula	Parámetros
W_Q, W_K, W_V, W_O	$4d^2$	67 108 864
SwiGLU	$3d\,d_{\text{ff}}$	135 266 304
Dos RMSNorm	$2d$	8 192
Bloque		202 383 360

5.3 Memoria

Definición 5.6 (Memoria de los parámetros). Si cada parámetro se almacena con b bytes, los pesos ocupan Nb bytes. Se usan $b = 4$ en fp32, $b = 2$ en bf16 y fp16, $b = 1$ en int8 y $b = 1/2$ en int4 (módulos 11 y 19). En este módulo, 1 GB $= 10^9$ bytes y 1 GiB $= 2^{30}$ bytes.

Proposición 5.7 (Estado del entrenamiento con Adam en precisión mixta). *Entrenar con Adam en precisión mixta requiere, sin contar las activaciones,* 16 *bytes por parámetro.*

Demostración. Por cada parámetro se almacenan: el peso de trabajo en bf16 (2 bytes), su gradiente en bf16 (2), la copia maestra del peso en fp32, que evita que las actualizaciones pequeñas se pierdan por redondeo (módulo 11, tema 5) (4), y los dos momentos de Adam m y v en fp32 (módulo 7) $(4 + 4)$. En total, $2 + 2 + 4 + 4 + 4 = 16$. ■

Cuadro 5.2: Memoria de LLaMA-2 7B ($N = 6\,738\,415\,616$) según el uso.

Uso	Bytes por parámetro	Memoria
Inferencia en bf16	2	13,5 GB
Inferencia en int4 (módulo 19)	0,5	3,4 GB
Pesos en fp32	4	27,0 GB
Entrenamiento, Adam en precisión mixta	16	107,8 GB

La cuadro 5.2 explica por qué un modelo de $7 \cdot 10^9$ parámetros cabe en una GPU de 24 GB para la inferencia en bf16, pero no para el entrenamiento: con 16 bytes por parámetro, en 24 GB solo caben los estados de un modelo de $1{,}5 \cdot 10^9$ parámetros, y aún habría que añadir las activaciones. Las técnicas ZeRO y FSDP (módulo 17) reparten estos 16 bytes entre varias GPU.

Observación 5.8 (Conexión con los LLM). Las cifras de esta sección se aplican a cualquier modelo cuya configuración sea pública, como GPT-2, LLaMA o Grok-1 [18]. Anthropic y Google no publican la configuración de Claude ni de Gemini, de modo que su número de parámetros no puede calcularse con estas fórmulas.

Lecturas recomendadas para este tema

- Kaplan *et al.* [9], §2.1 «Parameter and Compute Scaling of Transformers» y tabla 1.
- Jurafsky y Martin [8], §7.1.1 (coste de la atención multicabeza) y §7.8 «Current models and some numbers».
- Touvron *et al.* [15], §2.2 «Architecture» y tabla 2.
- Hashimoto y Liang [6], problemas `transformer_accounting` y `adamw_accounting`.

Ejercicios del tema 5

Ejercicio 5.1 ⋆⋆ Un Transformer tiene L capas, dimensión d, A cabezas con $d_k = d_v = d/A$ y $d_{\text{ff}} = 4d$. (a) Cuente los parámetros de W_Q, W_K, W_V y W_O en una capa. ¿Depende el resultado de A? (b) Cuente los parámetros de una capa MLP. (c) Demuestre que la pila de L bloques tiene unos $12Ld^2$ parámetros, sin contar sesgos ni normalizaciones. (d) Aplíquelo a $d = 512$ y $L = 6$.

Fuente: adaptado de Jurafsky y Martin [8], cap. 7, ejercicio 7.1 (se añade el apartado (d)).

Ejercicio 5.2 ⋆⋆ GPT-2 *small*: $d = 768$, $L = 12$, $V = 50\,257$, $n_{\text{ctx}} = 1024$, posiciones aprendidas, MLP con $d_{\text{ff}} = 4d$, LayerNorm con ganancia y sesgo, sesgos en todas las capas lineales y embeddings atados. Calcule el número de parámetros de un bloque y del modelo. ¿Qué fracción cubre $12Ld^2$?

Ejercicio 5.3 ⋆⋆ LLaMA-2 13B: $d = 5120$, $L = 40$, $d_{\text{ff}} = 13\,824$, $V = 32\,000$, sin atar. Calcule N exactamente y compárelo con $12Ld^2$.

Ejercicio 5.4 ⋆⋆ Para LLaMA-2 13B (N del ejercicio 5.3), calcule la memoria de los pesos en bf16 y la del estado completo de entrenamiento con Adam en precisión mixta, sin activaciones. ¿Cuántas GPU de 80 GB hacen falta como mínimo para el estado de entrenamiento?

Ejercicio 5.5 ⋆⋆ Un modelo con la arquitectura de este módulo (pre-norm, RMSNorm con ganancia, SwiGLU, RoPE, sin sesgos, embeddings sin atar) tiene la forma de GPT-2 XL: $V = 50\,257$, $n_{\text{ctx}} = 1024$, $L = 48$, $d = 1600$, $h = 25$ y $d_{\text{ff}} = 4288$. Cuente sus parámetros entrenables y la memoria que ocupan en fp32.

Fuente: adaptado de Hashimoto y Liang [6], problema `transformer_accounting`, apartado (a).

Ejercicio 5.6 ⋆ Una GPU tiene 24 GB. Sin contar activaciones, ¿cuál es el mayor número de parámetros de un modelo que cabe en ella (a) para la inferencia en bf16, (b) para la inferencia en int4 y (c) para el entrenamiento con Adam en precisión mixta? ¿Qué componentes del estado de entrenamiento no son necesarios para la inferencia?

Ejercicio 5.7 ⋆⋆⋆ Sea $f = 2Vd/N$ la fracción de parámetros de embeddings de un modelo sin atar con $d_{\text{ff}} = \frac{8}{3}d$ (corolario 5.4). (a) Demuestre que, si V y la razón $\alpha = L/d$ están fijos, $f \sim V/(6\alpha d^2)$ cuando $d \to \infty$. (b) Calcule f para LLaMA-2 7B y para el modelo del ejercicio 5.5 y explique por qué el atado de embeddings se abandona en los modelos grandes.

Tema 6

Contar el cómputo: FLOPs y la regla $6ND$

El coste de un Transformer se mide en operaciones de coma flotante (FLOPs). Casi todas proceden de productos de matrices, de modo que basta un único lema para contarlas. Este tema deduce el coste de una pasada hacia delante, unos $2N$ FLOPs por token más un término de atención proporcional al contexto, y el coste del entrenamiento, $C \approx 6ND$. Se sigue [9, §2.1], la sección de contabilidad de recursos de la práctica 1 de CS336 [6, §3.5] y los datos de entrenamiento de Llama 2 [16, §2.2.1 y tabla 2].

6.1 El coste de un producto de matrices

Definición 6.1 (FLOP). Un *FLOP* es una suma o una multiplicación en coma flotante. El *rendimiento* de un acelerador se mide en FLOP/s; su *aprovechamiento* (MFU, *model FLOPs utilization*) es el cociente entre los FLOP/s útiles que alcanza un entrenamiento y el máximo teórico del hardware [6, problema `adamw_accounting`].

Lema 6.2 (Regla $2mkp$). *El producto de una matriz $m \times k$ por una matriz $k \times p$ requiere $2mkp$ FLOPs, contando como $2k$ las operaciones de cada producto escalar de longitud k.*

Demostración. El resultado tiene mp entradas, y cada una es un producto escalar $\sum_{r=1}^{k} a_r b_r$ que requiere k multiplicaciones y k sumas de acumulación (o $k-1$ si no se cuenta la primera, que no altera el orden de magnitud). En total, $mp \cdot 2k$. ∎

6.2 La pasada hacia delante

Proposición 6.3 (Coste hacia delante). *Sea N_{mat} el número de parámetros de las matrices que multiplican al flujo (proyecciones de atención, MLP y unembedding). Una pasada hacia delante sobre una secuencia de n tokens requiere*

$$C_{\text{fwd}} = 2nN_{\text{mat}} + 4Ln^2d$$

FLOPs, más operaciones elemento a elemento (normalizaciones, softmax, activaciones, RoPE) de coste $O(Lnd + Ln^2h)$, que se desprecian. Por token, el coste es $2N_{\text{mat}} + 4Lnd \approx 2N + 4Lnd$.

Demostración. Cada matriz de pesos $W \in \mathbb{R}^{a \times b}$ multiplica a una matriz $n \times a$; por el lema 6.2, cuesta $2nab$, es decir, $2n$ FLOPs por peso. Sumando sobre todas las matrices se obtiene $2nN_{\text{mat}}$. El embedding de entrada es una indexación y no cuesta FLOPs. Quedan los dos productos de la atención que no involucran pesos: en cada capa y cada cabeza, $Q_i K_i^\top$ es $(n \times d_k)(d_k \times n)$ y cuesta $2n^2 d_k$, y el producto de los pesos por V_i es $(n \times n)(n \times d_k)$ y cuesta lo mismo. Con h

cabezas y $hd_k = d$, son $4n^2d$ por capa y $4Ln^2d$ en total. Dividiendo por n se obtiene el coste por token; N_{mat} difiere de N solo en el embedding de entrada y las normalizaciones. ■

Observación 6.4. Kaplan *et al.* [9, §2.1, ecuación (2.2)] escriben $C_{\text{fwd}} \approx 2N + 2Ln_{\text{ctx}}d$ por token. El factor es la mitad del de la proposición 6.3, lo que concuerda con tener en cuenta la máscara causal: el token t solo atiende a t posiciones, y el promedio de t sobre la secuencia es aproximadamente $n/2$. La práctica de CS336 cuenta los productos completos $n \times n$, como la proposición 6.3.

Corolario 6.5 (Contexto en el que la atención iguala a los pesos). *El término de atención por token, $4Lnd$, iguala a $2N$ cuando $n^* = N/(2Ld)$.*

Demostración. Se despeja n en $4Lnd = 2N$. ■

Para LLaMA-2 7B, $n^* = 6\,738\,415\,616/(2 \cdot 32 \cdot 4096) \approx 25\,705$ tokens; con su contexto de 4096 tokens, la atención añade un 15,9 % al coste $2N$. Para contextos mucho mayores domina el término cuadrático, lo que motiva FlashAttention (módulo 17) y la atención con ventana deslizante (módulo 18).

6.3 La pasada hacia atrás y la regla $6ND$

Teorema 6.6 (El retroceso cuesta el doble). *Sea $Y = XW$ una capa lineal con $X \in \mathbb{R}^{n\times a}$ y $W \in \mathbb{R}^{a\times b}$, y sea $G = \partial\mathcal{L}/\partial Y \in \mathbb{R}^{n\times b}$. La pasada hacia delante cuesta $2nab$ FLOPs y la pasada hacia atrás, que calcula*

$$\frac{\partial \mathcal{L}}{\partial X} = GW^\top \qquad y \qquad \frac{\partial \mathcal{L}}{\partial W} = X^\top G,$$

cuesta $4nab$ FLOPs.

Demostración. Las dos fórmulas de los gradientes son las de la retropropagación por una capa lineal (módulo 10). Por el lema 6.2: $Y = XW$ es $(n \times a)(a \times b)$, con $2nab$ FLOPs; $GW^\top$ es $(n \times b)(b \times a)$, con $2nba$; $X^\top G$ es $(a \times n)(n \times b)$, con $2anb$. El primer gradiente hace falta para propagar hacia las capas anteriores y el segundo para actualizar W. ■

Corolario 6.7 (Regla $6ND$). *Entrenar un Transformer de N parámetros con D tokens cuesta $C \approx 6ND$ FLOPs, despreciando el término de atención y las operaciones elemento a elemento.*

Demostración. Por el teorema 6.6, cada peso de una capa lineal cuesta $2 + 4 = 6$ FLOPs por token entre las dos pasadas. Sumando sobre todos los pesos y todos los tokens, $6ND$. La actualización de Adam cuesta $O(N)$ FLOPs por paso, despreciables frente a los $O(N \cdot$ tokens por paso) de las dos pasadas. ■

Kaplan *et al.* enuncian la misma estimación, $C \approx 6N$ FLOPs por token de entrenamiento [9, §2.1], que es la base de las leyes de escalado del módulo 16.

Ejemplo 6.8 (Coste de LLaMA-2 7B). Llama 2 7B se entrenó con $D = 2 \cdot 10^{12}$ tokens [16, §2.1]. Por el corolario 6.7,

$$C \approx 6 \cdot 6{,}738 \cdot 10^9 \cdot 2 \cdot 10^{12} = 8{,}086 \cdot 10^{22} \text{ FLOPs}.$$

Una GPU A100 tiene un máximo de $312 \cdot 10^{12}$ FLOP/s en bf16. Con un aprovechamiento del 40 %, el entrenamiento requeriría $8{,}086 \cdot 10^{22} / (0{,}4 \cdot 312 \cdot 10^{12}) \approx 6{,}48 \cdot 10^{8}$ s, unas 180 000 horas de GPU. El artículo declara 184 320 horas de A100-80GB para el modelo de 7B [16, tabla 2], lo que corresponde a un aprovechamiento del 39 %.

Observación 6.9 (Conexión con los LLM). La regla $6ND$ permite estimar el coste de cualquier entrenamiento cuyos N y D sean públicos. Para Claude, Gemini y Grok, sus desarrolladores no han publicado D, y en los modelos con mezcla de expertos (módulo 18) el coste depende de los parámetros *activos* por token, no del total.

Lecturas recomendadas para este tema

- Kaplan *et al.* [9], §2.1 y tabla 1 (parámetros y cómputo por operación).
- Hashimoto y Liang [6], §3.5, apartado «Resource accounting», y los problemas de contabilidad `transformer_accounting` y `adamw_accounting`.
- Touvron *et al.* [16], §2.2.1 y tabla 2 (horas de GPU de Llama 2).

Ejercicios del tema 6

Ejercicio 6.1 ⋆ Explique por qué una pasada hacia delante cuesta unos 2 FLOPs por parámetro y por token. Compruébelo contando las operaciones del producto de un vector de $\mathbb{R}^2$ por una matriz 2×3.

Ejercicio 6.2 ⋆⋆ LLaMA-2 7B ($N = 6{,}738 \cdot 10^9$) se entrenó con $2 \cdot 10^{12}$ tokens. Calcule los FLOPs de entrenamiento y las horas de A100 ($312 \cdot 10^{12}$ FLOP/s) con un aprovechamiento del 40 %. Despeje el aprovechamiento real a partir de las 184 320 horas declaradas.

Ejercicio 6.3 ⋆⋆ Considere la arquitectura del ejercicio 5.5 con la forma de GPT-2 *small* ($L = 12$, $d = 768$, $h = 12$, $d_{\text{ff}} = 2048$, $V = 50\,257$) y una secuencia de $n = 1024$ tokens. (a) Enumere los productos de matrices de un bloque y su coste con el lema 6.2. (b) Calcule el coste total de la pasada hacia delante. (c) ¿Qué parte domina? (d) Compare las proporciones con las del modelo con forma de GPT-2 XL que se dan en la solución.

Fuente: adaptado de Hashimoto y Liang [6], problema `transformer_accounting`, apartados (b)–(d).

Ejercicio 6.4 ⋆⋆ Repita el recuento de la pasada hacia delante del modelo del ejercicio 5.5 con $n = 16\,384$ en lugar de 1024. ¿Por qué factor se multiplica el total? ¿Qué fracción pasa a corresponder a los productos $QK^\top$ y AV?

Fuente: adaptado de Hashimoto y Liang [6], problema `transformer_accounting`, apartado (e).

Ejercicio 6.5 ⋆⋆ Se entrena el modelo del ejercicio 5.5 durante 400 000 pasos con lotes de 1024 secuencias de 1024 tokens en una sola GPU H100, cuyo máximo en TF32 es $495 \cdot 10^{12}$ FLOP/s, con un aprovechamiento del 50 %. Supóngase que la pasada hacia atrás cuesta el doble que la pasada hacia delante. ¿Cuántas horas tarda? Compárelo con la estimación $6ND$.

Fuente: adaptado de Hashimoto y Liang [6], problema `adamw_accounting`, apartado (d).

Ejercicio 6.6 ⋆⋆⋆ Demuestre el teorema 6.6 para una capa $Y = XW + \mathbf{1}c^\top$ con sesgo $c \in \mathbb{R}^b$: obtenga los tres gradientes, cuente sus FLOPs y demuestre que el coste del sesgo es despreciable frente al de W cuando a es grande. Compruebe el recuento con $n = 1$ y $a = b = 2$ sin sesgo.

Ejercicio 6.7 ⋆⋆⋆ Para LLaMA-2 7B con $n = 4096$: (a) calcule la fracción que representa el término de atención $4Lnd$ respecto de $2N$; (b) obtenga n^* del corolario 6.5; (c) demuestre que la fracción es lineal en n y compruebe la coherencia de (a) y (b).

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; los recuentos de parámetros y de FLOPs se dan exactos. Todas las cifras se han comprobado con un programa en Python.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. $V = 100$, $d = 8$, $h = 2$, $d_{\text{ff}} = 32$, $n = 5$.

Planteamiento. seguir la definición 1.3 y la proposición 1.4; la dimensión por cabeza es $d_k = d/h$.

Desarrollo.

1. (a) La entrada es un vector de $n = 5$ enteros en $\{1, \dots, 100\}$.
2. (b) $H^0 = OW_E$ con $O \in \{0,1\}^{5\times 100}$ y $W_E \in \mathbb{R}^{100\times 8}$: forma 5×8.
3. (c) $d_k = 8/2 = 4$; $Q_i = ZW_Q^{(i)}$ con $W_Q^{(i)} \in \mathbb{R}^{8\times 4}$: forma 5×4.
4. (d) $\text{softmax}(Q_i K_i^\top/\sqrt{d_k} + M)$: $(5 \times 4)(4 \times 5)$, forma 5×5, con ceros por encima de la diagonal.
5. (e) XW_g y XW_u con $W_g, W_u \in \mathbb{R}^{8\times 32}$: forma 5×32.
6. (f) $Z = \text{Norm}_f(H^L)W_U$ con $W_U \in \mathbb{R}^{8\times 100}$: forma 5×100.

Resultado. **(a) 5 enteros; (b)** 5×8**; (c)** 5×4**; (d)** 5×5**; (e)** 5×32**; (f)** 5×100**.** Una pasada produce **5 distribuciones**, una por fila de Z (corolario 1.6).

Comprobación. todas las matrices del flujo tienen forma 5×8, como exige la proposición 1.4; las dos cabezas de 5×4 concatenadas dan de nuevo 5×8.

Solución del ejercicio 1.2

Datos. $W_E \in \mathbb{R}^{V\times d}$; el unembedding necesita una matriz $d \times V$; $V = 50\,257$, $d = 768$.

Planteamiento. aplicar la definición 1.7 y la proposición 1.8.

Desarrollo.

1. Se comparte W_E, de forma $V \times d$; la salida usa su traspuesta, $W_U = W_E^\top$, de forma $d \times V$.
2. El logit del token j es $z_j = h \cdot e_j$: el vector final del flujo se compara con el embedding de cada token.

3. Si h se parece al embedding de un token, ese token recibe un logit alto. Tokens que aparecen en contextos parecidos deben tener a la vez embeddings parecidos y plantillas de salida parecidas, de modo que usar los mismos vectores es coherente.

4. Ahorro: $Vd = 50\,257 \cdot 768 = 38\,597\,376$ parámetros.

Resultado. **se comparte $W_E \in \mathbb{R}^{V\times d}$; en GPT-2 *small* se ahorran 38 597 376 parámetros.**

Comprobación. es el 31,0 % del total de 124 439 808 parámetros calculado en el ejercicio 5.2: $38\,597\,376/124\,439\,808 = 0{,}3102$.

Solución del ejercicio 1.3

Datos. $e_1 = (1, 0)$, $e_2 = (0, 1)$, $e_3 = (1, 1)$; $h = (2, 1)$; embeddings atados.

Planteamiento. $z_j = h \cdot e_j$ y $p = \text{softmax}(z)$.

Desarrollo.

1. $z_1 = 2 \cdot 1 + 1 \cdot 0 = 2$; $z_2 = 2 \cdot 0 + 1 \cdot 1 = 1$; $z_3 = 2 \cdot 1 + 1 \cdot 1 = 3$.

2. $e^2 = 7{,}389$; $e^1 = 2{,}718$; $e^3 = 20{,}09$; suma $= 30{,}19$.

3. $p = (7{,}389;\ 2{,}718;\ 20{,}09)/30{,}19 = (0{,}2447;\ 0{,}0900;\ 0{,}6652)$.

4. Por la proposición 1.8, el ganador maximiza $\|e_j\| \cos\angle(h, e_j) = z_j/\|h\|$. Con $\|h\| = \sqrt{5} = 2{,}236$: 0,8944, 0,4472 y 1,342.

Resultado. $z = (2, 1, 3)$ **y** $p = (0{,}2447;\ 0{,}0900;\ 0{,}6652)$**.** El token 3 es el más probable: su embedding forma un ángulo pequeño con h (cos = 0,9487) y además tiene la mayor norma, $\sqrt{2}$.

Comprobación. $0{,}2447 + 0{,}0900 + 0{,}6652 = 0{,}9999 \approx 1$, por redondeo; el orden de p coincide con el de z, como exige la monotonía de la softmax (módulo 1).

Solución del ejercicio 1.4

Datos. $n = 4$ y $n = 1024$; fila $t = 3$ con $s_3 = (1, 2, 0, 5)$.

Planteamiento. en la fila t solo son finitas las entradas $u \le t$ (proposición 1.5).

Desarrollo.

1. (a) La fila t tiene t entradas permitidas: $1 + 2 + 3 + 4 = 10$ pesos no nulos posibles.

2. En general, $n(n + 1)/2$ de las n^2 entradas; la fracción es $(n + 1)/(2n)$. Con $n = 1024$: $1025/2048 = 0{,}5005$.

3. (b) Para $t = 3$ se descarta $u = 4$: $s_3 + M_3 = (1, 2, 0, -\infty)$.

4. $e^1 = 2{,}718$, $e^2 = 7{,}389$, $e^0 = 1$, $e^{-\infty} = 0$; suma $= 11{,}11$.

5. Pesos: $(2{,}718;\ 7{,}389;\ 1;\ 0)/11{,}11 = (0{,}2447;\ 0{,}6652;\ 0{,}0900;\ 0)$.

Resultado. **(a) 10 pesos, y una fracción de** 0,5005 **con** $n = 1024$**. (b) Los pesos de la fila 3 son** $(0{,}2447;\ 0{,}6652;\ 0{,}0900;\ 0)$**.**

Comprobación. los pesos suman 1, y son los mismos números del ejercicio 1.3 reordenados, porque las puntuaciones finitas son de nuevo $\{0, 1, 2\}$.

Solución del ejercicio 1.5

Datos. fila $s_3 = (1, 2, 0, 5)$ del ejercicio 1.4.

Planteamiento. la máscara se *suma* a las puntuaciones antes de la softmax; comparar el efecto de sumar $-\infty$ y de sumar 0.

Desarrollo.

1. Sumar 0 no modifica las puntuaciones: la softmax se calcula sobre $(1, 2, 0, 5)$ y la posición futura sigue presente.
2. $e^5 = 148{,}4$; suma $= 2{,}718 + 7{,}389 + 1 + 148{,}4 = 159{,}5$.
3. Pesos: (0,0170; 0,0463; 0,0063; 0,9304).
4. Con $-\infty$, en cambio, $e^{-\infty} = 0$ y el peso futuro es exactamente nulo (ejercicio 1.4).

Resultado. **con 0, el token futuro recibe un peso de** 0,9304: el modelo podría copiar la respuesta que debe predecir. La máscara debe usar $-\infty$ (en la práctica, el menor número representable) para que la softmax asigne peso nulo.

Comprobación. la alternativa de multiplicar los pesos por una máscara de ceros y unos después de la softmax anularía la entrada, pero la fila dejaría de sumar 1; sumar $-\infty$ antes de la softmax anula y normaliza a la vez.

Solución del ejercicio 1.6

Datos. $M_{tu} = 0$ si $u \le t$ y $-\infty$ si $u > t$; $(PMP^\top)_{tu} = M_{\pi(t)\pi(u)}$.

Planteamiento. contar las entradas finitas de cada fila, un número que la permutación de columnas no altera.

Desarrollo.

1. La fila t de M tiene exactamente t entradas finitas (las de $u \le t$).
2. La fila t de $PMP^\top$ es la fila $\pi(t)$ de M con las columnas reordenadas por π; reordenar no cambia cuántas entradas son finitas. Luego tiene $\pi(t)$ entradas finitas.
3. Si $PMP^\top = M$, las filas t de ambas matrices tienen el mismo número de entradas finitas: $\pi(t) = t$ para todo t, es decir, π es la identidad.
4. Recíprocamente, si π es la identidad, $P = I$ y $PMP^\top = M$.

Resultado. $PMP^\top = M$ **si y solo si** $\pi = \mathrm{id}$**.** La máscara distingue todas las posiciones: la fila t es la única que ve exactamente t tokens. Por eso un decoder causal sin codificación de posición ya no es equivariante por permutaciones y puede, en principio, inferir la posición, como señala la práctica de CS336 [6, §7.3].

Comprobación. con $n = 2$ y π el intercambio, $PMP^\top = \left(\begin{smallmatrix} 0 & 0 \\ -\infty & 0 \end{smallmatrix}\right) \neq M = \left(\begin{smallmatrix} 0 & -\infty \\ 0 & 0 \end{smallmatrix}\right)$.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. un bloque con entrada x, subcapas MHA y MLP y normalizaciones Norm_1, Norm_2.

Planteamiento. escribir las dos variantes y seguir el término x hasta la salida.

Desarrollo.

1. Pre-norm: $a = x + \mathrm{MHA}(\mathrm{Norm}_1(x))$ y $y = a + \mathrm{MLP}(\mathrm{Norm}_2(a))$.
2. Sustituyendo: $y = x + \mathrm{MHA}(\mathrm{Norm}_1(x)) + \mathrm{MLP}(\mathrm{Norm}_2(a))$. El término x aparece sumado sin modificar.
3. Post-norm: $a = \mathrm{Norm}_1(x + \mathrm{MHA}(x))$ y $y = \mathrm{Norm}_2(a + \mathrm{MLP}(a))$.
4. Sustituyendo: $y = \mathrm{Norm}_2\big(\mathrm{Norm}_1(x + \mathrm{MHA}(x)) + \mathrm{MLP}(\cdots)\big)$. La entrada x pasa por Norm_1, que la divide por un RMS, y el resultado vuelve a reescalarse en Norm_2. Ningún sumando de y es x sin modificar.

Resultado. **en pre-norm** existe un camino identidad $x \to y$ que no atraviesa ninguna normalización; en post-norm toda la señal atraviesa las dos normalizaciones.

Comprobación. con L bloques, pre-norm da $H^L = H^0 + \sum_k F_k(\cdot)$ (proposición 2.2), mientras que en post-norm $\mathrm{RMS}(h_t^L) = 1$ sea cual sea la entrada (proposición 2.6), de modo que H^L no puede escribirse como H^0 más algo.

Solución del ejercicio 2.2

Planteamiento. comparar la escala de h_t^L en las dos variantes (proposiciones 2.6 y 2.7).

Desarrollo.

1. En post-norm, la salida de cada bloque es la salida de una normalización: h_t^L ya tiene RMS controlado (igual a 1 si no hay ganancia) antes del unembedding.
2. En pre-norm, $h_t^L = h_t^0 + \sum_k F_k(\cdot)_t$ y su norma cuadrática media crece linealmente con el número de subcapas (proposición 2.7).
3. Los logits $z = h_t^L W_U$ escalan con $\|h_t^L\|$; sin normalizar, su escala y, por tanto, la temperatura efectiva de la softmax (módulo 1) dependerían de la profundidad y cambiarían durante el entrenamiento.

Resultado. **pre-norm necesita** Norm_f **porque el flujo no está normalizado en ningún punto y su norma crece; en post-norm la última operación de la pila ya es una normalización.**

Comprobación. es la capa adicional que señalan Jurafsky y Martin [8, §7.2.3] y la que añade GPT-2 tras el último bloque [12, §2.3].

Solución del ejercicio 2.3

Datos. $h^0 = (1, 0)$; aportaciones $(0;\ 0{,}5)$ y $(-0{,}2;\ 0{,}3)$; $d = 2$; $\mathrm{Norm}(x) = x/\,\mathrm{RMS}(x)$.

Planteamiento. calcular el RMS de cada vector del flujo y dividir; proyectar cada aportación sobre $u = (0, 1)$.

Desarrollo.

1. (a) $\mathrm{RMS}(h^0) = \sqrt{(1^2 + 0^2)/2} = \sqrt{0{,}5} = 0{,}7071$; $\mathrm{Norm}(h^0) = (1{,}414;\ 0)$.
2. (b) $a^1 = (1;\ 0{,}5)$; $\mathrm{RMS}(a^1) = \sqrt{(1 + 0{,}25)/2} = \sqrt{0{,}625} = 0{,}7906$; $\mathrm{Norm}(a^1) = (1{,}265;\ 0{,}6325)$.
3. (c) $h^1 = (0{,}8;\ 0{,}8)$; $\mathrm{RMS}(h^1) = 0{,}8$; $\mathrm{Norm}(h^1) = (1;\ 1)$.
4. (d) Componentes sobre u: $h^0 \cdot u = 0$; atención 0,5; MLP 0,3.

Resultado. **(a)** (1,414; 0)**; (b)** (1,265; 0,6325)**; (c)** (1; 1)**; (d)** 0**,** 0,5 **y** 0,3**.**

Comprobación. $0 + 0{,}5 + 0{,}3 = 0{,}8 = h^1 \cdot u$, como predice la proposición 2.2; cada vector normalizado tiene RMS 1, por ejemplo $\sqrt{(1{,}265^2 + 0{,}6325^2)/2} = \sqrt{(1{,}600 + 0{,}4000)/2} = 1{,}000$.

Solución del ejercicio 2.4

Datos. $L = 32$, luego $2L = 64$ subcapas; $\mathbb{E}\|h_t^0\|^2 = 1$; $s^2 = 0{,}5$; objetivo 1,5.

Planteamiento. aplicar la proposición 2.7; multiplicar los pesos de salida por c multiplica cada aportación por c y s^2 por c^2.

Desarrollo.

1. (a) $\mathbb{E}\|h_t^{32}\|^2 = 1 + 64 \cdot 0{,}5 = 33$; cociente $33/1 = 33$ (la norma típica crece en un factor $\sqrt{33} = 5{,}745$).
2. (b) Se exige $1 + 64 \cdot 0{,}5\, c^2 = 1{,}5$, es decir, $32c^2 = 0{,}5$.
3. $c^2 = 0{,}5/32 = 1/64$; $c = 1/8 = 0{,}125$.

Resultado. **(a)** 33**; (b)** $c = 1/8$, que es exactamente $1/\sqrt{2L} = 1/\sqrt{64}$, el factor de GPT-2 con $N = 2L$ capas residuales.

Comprobación. con $c = 1/8$, cada subcapa aporta $0{,}5/64 = 0{,}0078$ y las 64 suman 0,5: $1 + 0{,}5 = 1{,}5$.

Solución del ejercicio 2.5

Datos. $\partial H^L/\partial H^0 = \prod_{k=2L}^{1}(I + J_k)$ con $\|J_k\|_2 \le \varepsilon$.

Planteamiento. desarrollar el producto como en la proposición 2.4 y acotar cada término con la submultiplicatividad de la norma.

Desarrollo.

1. Al multiplicar los $2L$ binomios se obtiene $I + \sum_S J_S$, donde S recorre los subconjuntos no vacíos de $\{1, \ldots, 2L\}$ y J_S es el producto de los J_k con $k \in S$ en orden decreciente.
2. Por la desigualdad triangular y $\|AB\|_2 \le \|A\|_2\|B\|_2$: $\|J_S\|_2 \le \varepsilon^{|S|}$.
3. Hay $\binom{2L}{j}$ subconjuntos de tamaño j; luego $\|\partial H^L/\partial H^0 - I\|_2 \le \sum_{j=1}^{2L}\binom{2L}{j}\varepsilon^j = (1 + \varepsilon)^{2L} - 1$, por el binomio de Newton.
4. Como $1 + \varepsilon \le e^{\varepsilon}$, $(1 + \varepsilon)^{2L} - 1 \le e^{2L\varepsilon} - 1$.
5. La función $x \mapsto e^x - 1$ es convexa y vale 0 en $x = 0$ y $e - 1$ en $x = 1$; en $[0, 1]$ queda por debajo de la cuerda: $e^x - 1 \le (e - 1)x$. Con $x = 2L\varepsilon \le 1$ se obtiene la cota $(e - 1)\, 2L\varepsilon$.

Resultado. $\|\partial H^L/\partial H^0 - I\|_2 \le (1+\varepsilon)^{2L} - 1 \le (e-1)\, 2L\varepsilon$ **si** $2L\varepsilon \le 1$**.** Si cada subcapa es una perturbación pequeña del flujo, la red completa se comporta, a efectos del gradiente, como la identidad más una corrección de orden $2L\varepsilon$: el gradiente no se desvanece ni explota. La inicialización escalada de GPT-2 busca precisamente que ε decrezca con la profundidad.

Comprobación. con $L = 32$ y $\varepsilon = 0{,}01$: $(1{,}01)^{64} - 1 = 0{,}8905$ y $(e-1) \cdot 0{,}64 = 1{,}100$; la primera cota es menor que la segunda, como debe ser.

A.3 Tema 3

Solución del ejercicio 3.1

Datos. $d = 512$, $d_{\text{ff}} = 2048$, con sesgos $b_1 \in \mathbb{R}^{2048}$ y $b_2 \in \mathbb{R}^{512}$.

Planteamiento. aplicar la definición 3.1 y el lema 3.2.

Desarrollo.

1. El MLP calcula $\phi(xW_1 + b_1)W_2 + b_2$ para cada fila x por separado y con los mismos parámetros: la fila t de la salida solo depende de la fila t de la entrada, de modo que no hay intercambio de información entre tokens.
2. Matrices: $2 \cdot 512 \cdot 2048 = 2\,097\,152$.
3. Sesgos: $2048 + 512 = 2560$.
4. Total: $2\,097\,152 + 2560 = 2\,099\,712$.
5. Fracción de sesgos: $2560/2\,099\,712 = 0{,}001219$.

Resultado. $2\,099\,712$ **parámetros; los sesgos son el** $0{,}12\,\%$**.** Por eso LLaMA y la práctica de CS336 los omiten sin pérdida apreciable de capacidad.

Comprobación. $8d^2 = 8 \cdot 262\,144 = 2\,097\,152$ coincide con la parte matricial (proposición 3.6).

Solución del ejercicio 3.2

Datos. MLP clásico con $d_{\text{ff}} = 4d$; SwiGLU con tres matrices; $d = 4096$.

Planteamiento. igualar parámetros (proposición 3.6) y redondear hacia arriba a múltiplo de 256.

Desarrollo.

1. MLP clásico: $d \cdot 4d + 4d \cdot d = 8d^2$. SwiGLU: $3d\,d_{\text{ff}}$. Igualdad: $d_{\text{ff}} = 8d/3$.
2. $8 \cdot 4096/3 = 32\,768/3 = 10\,922{,}67$.
3. $10\,922{,}67/256 = 42{,}67$; el entero siguiente es 43 y $43 \cdot 256 = 11\,008$.
4. SwiGLU de LLaMA-2: $3 \cdot 4096 \cdot 11\,008 = 135\,266\,304$. Referencia: $8 \cdot 4096^2 = 8 \cdot 16\,777\,216 = 134\,217\,728$.
5. Exceso: $135\,266\,304 - 134\,217\,728 = 1\,048\,576$.

Resultado. $d_{\text{ff}} = 11\,008$**; el MLP tiene** $1\,048\,576 = 2^{20}$ **parámetros más por bloque que** $8d^2$**, un** $0{,}78\,\%$**.**

Comprobación. $3 \cdot 4096 \cdot (11\,008 - 32\,768/3) = 12\,288 \cdot 11\,008 - 4096 \cdot 32\,768 = 135\,266\,304 - 134\,217\,728 = 1\,048\,576$.

Solución del ejercicio 3.3

Datos. $x = (1, -1)$; W_g, W_u, W_{d} del enunciado; $\sigma(1) = 0{,}7311$, luego $\sigma(-1) = 1 - 0{,}7311 = 0{,}2689$.

Planteamiento. calcular la puerta xW_g, la rama lineal xW_u, la capa intermedia $\text{SiLU}(xW_g) \odot xW_u$ y multiplicar por W_{d}.

Desarrollo.

1. Puerta: $xW_g = (1 \cdot 2 + (-1) \cdot 1;\ 1 \cdot 0 + (-1) \cdot 1) = (1;\ -1)$.
2. $\mathrm{SiLU}(1) = 1 \cdot 0{,}7311 = 0{,}7311$; $\mathrm{SiLU}(-1) = -1 \cdot 0{,}2689 = -0{,}2689$.
3. Rama lineal: $xW_u = (1 \cdot 1 + (-1) \cdot 0;\ 1 \cdot 1 + (-1) \cdot 2) = (1;\ -1)$.
4. Capa intermedia: $(0{,}7311 \cdot 1;\ (-0{,}2689) \cdot (-1)) = (0{,}7311;\ 0{,}2689)$.
5. Salida: $(0{,}7311 \cdot 1 + 0{,}2689 \cdot 2;\ 0{,}7311 \cdot (-1) + 0{,}2689 \cdot 0) = (1{,}269;\ -0{,}7311)$.

Resultado. $\mathrm{SwiGLU}(x) = (1{,}269;\ -0{,}7311)$, con puerta $(1; -1)$, rama lineal $(1; -1)$ y capa intermedia $(0{,}7311;\ 0{,}2689)$.

Comprobación. las dos componentes de la capa intermedia suman 1 porque, con estos datos, valen $\sigma(1)$ y $\sigma(-1)$, y $\sigma(x) + \sigma(-x) = 1$. La segunda componente es positiva aunque la puerta sea negativa: el producto de dos ramas negativas da un valor positivo, algo imposible con ReLU.

Solución del ejercicio 3.4

Datos. $d = 1600$ y $d = 768$; SwiGLU con $3d\,d_{\mathrm{ff}}$ parámetros; $\mathrm{FFN}_{\mathrm{SiLU}}$ con $2d \cdot 4d$.

Planteamiento. dividir $\frac{8}{3}d$ por 64, redondear al entero más próximo y contar con la proposición 3.6.

Desarrollo.

1. (a) $\frac{8}{3} \cdot 1600 = 4266{,}67$; $4266{,}67/64 = 66{,}67$; el entero más cercano es 67 y $67 \cdot 64 = 4288$. Los candidatos vecinos distan $4266{,}67 - 4224 = 42{,}67$ y $4288 - 4266{,}67 = 21{,}33$.
2. (b) SwiGLU: $3 \cdot 1600 \cdot 4288 = 4800 \cdot 4288 = 20\,582\,400$.
3. $\mathrm{FFN}_{\mathrm{SiLU}}$ con $d_{\mathrm{ff}} = 6400$: $2 \cdot 1600 \cdot 6400 = 20\,480\,000$.
4. (c) Cociente: $20\,582\,400/20\,480\,000 = 1{,}005$.
5. (d) $\frac{8}{3} \cdot 768 = 2048 = 32 \cdot 64$, múltiplo exacto. SwiGLU: $3 \cdot 768 \cdot 2048 = 4\,718\,592$; $\mathrm{FFN}_{\mathrm{SiLU}}$ con $d_{\mathrm{ff}} = 3072$: $2 \cdot 768 \cdot 3072 = 4\,718\,592$.

Resultado. **(a)** $d_{\mathrm{ff}} = 4288$**; (b)** $20\,582\,400$ **frente a** $20\,480\,000$**; (c)** $1{,}005$**; (d)** $d_{\mathrm{ff}} = 2048$ **y ambos MLP tienen exactamente** $4\,718\,592$ **parámetros.** La comparación del problema `swiglu_ablation` es, por tanto, justa: difiere en menos de un 0,5 % de parámetros.

Comprobación. con $d = 768$ se recupera la proporción de Shazeer: $d_{\mathrm{ff}} = 2048$ para la variante con puerta y $3072 = 4 \cdot 768$ para la de dos matrices [13, §3.1].

Solución del ejercicio 3.5

Datos. $\mathrm{SiLU}(x) = x\sigma(x)$; $\sigma' = \sigma(1 - \sigma)$; $\sigma(-x) = 1 - \sigma(x)$.

Planteamiento. (ii) y (iii) como en la proposición 3.4; para la identidad, expresar σ con la tangente hiperbólica.

Desarrollo.

1. (ii) Regla del producto: $\mathrm{SiLU}'(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x)) = \sigma(x)\big(1 + x(1 - \sigma(x))\big)$; en 0, $\sigma(0) = \frac{1}{2}$ y $\mathrm{SiLU}'(0) = \frac{1}{2}$.
2. (iii) Los puntos críticos son los ceros de $g(x) = 1 + x(1 - \sigma(x))$. Si $x \geq 0$, $g(x) \geq 1$. Si $x < 0$, $g'(x) = (1 - \sigma(x))(1 - x\sigma(x)) > 0$, $g(0) = 1$ y $g(x) \to -\infty$ cuando $x \to -\infty$; hay una única raíz $x^* < 0$, que es mínimo porque SiLU' pasa de negativa a positiva. De $g(x^*) = 0$: $\sigma(x^*) = 1 + 1/x^*$ y $\mathrm{SiLU}(x^*) = x^* + 1$.

3. $\tanh\frac{x}{2} = \dfrac{1-e^{-x}}{1+e^{-x}}$, luego $1+\tanh\frac{x}{2} = \dfrac{2}{1+e^{-x}} = 2\sigma(x)$ y $\mathrm{SiLU}(x) = \frac{x}{2}\big(1+\tanh\frac{x}{2}\big)$.
4. $\mathrm{SiLU}(x) - \mathrm{SiLU}(-x) = x\sigma(x) + x\sigma(-x) = x\big(\sigma(x) + 1 - \sigma(x)\big) = x$.
5. $\mathrm{SiLU}(x) + \mathrm{SiLU}(-x) = x\big(\sigma(x) - \sigma(-x)\big) = x\big(2\sigma(x) - 1\big) = x\tanh\frac{x}{2}$.
6. La parte impar de una función es $\frac{1}{2}\big(f(x) - f(-x)\big)$; para SiLU vale $x/2$.

Resultado. $\mathrm{SiLU}'(0) = \frac{1}{2}$**; mínimo único en** $x^* \approx -1{,}2785$ **con valor** $x^* + 1 \approx -0{,}2785$**;** $\mathrm{SiLU}(x) = \frac{x}{2}(1+\tanh\frac{x}{2})$**;** $\mathrm{SiLU}(x) - \mathrm{SiLU}(-x) = x$**.** SiLU es la recta $x/2$ más la función par $\frac{x}{2}\tanh\frac{x}{2} \geq 0$.

Comprobación. con $x = 1$: $\mathrm{SiLU}(1) - \mathrm{SiLU}(-1) = 0{,}7311 - (-0{,}2689) = 1$ y $\mathrm{SiLU}(1) + \mathrm{SiLU}(-1) = 0{,}4622 \approx \tanh(0{,}5) = 0{,}4621$.

A.4 Tema 4

Solución del ejercicio 4.1

Planteamiento. separar los dos papeles de la atención: decidir a quién se atiende $(q \cdot k)$ y qué se copia (v).

Desarrollo.

1. La puntuación $(R_m q)^\top (R_n k) = q^\top R_{n-m} k$ solo depende de la distancia $n - m$ (teorema 4.8); para ello basta girar q y k.
2. El valor v_n es el contenido que se suma al flujo residual. Si se girase según la posición n, el mismo token aportaría vectores distintos según dónde estuviera, y la suma ponderada mezclaría contenidos girados con ángulos diferentes.
3. Las capas siguientes tendrían que deshacer esas rotaciones para reconocer el contenido, un trabajo adicional sin ventaja.

Resultado. **girar** q **y** k **basta para que los pesos dependan de la distancia; girar** v **mezclaría la posición absoluta en el contenido copiado.**

Comprobación. el artículo original [14], el código de LLaMA [10] y la práctica de CS336 [6, §3.4.3] aplican RoPE solo a consultas y claves.

Solución del ejercicio 4.2

Datos. $d = 4$; $\omega_0 = 1$, $\omega_1 = 10000^{-2/4}$; ángulos en radianes.

Planteamiento. aplicar la definición 4.2 y redondear a cuatro decimales.

Desarrollo.

1. $\omega_0 = 10000^0 = 1$; $\omega_1 = 10000^{-1/2} = 1/100 = 0{,}01$.
2. $p = 1$: $\sin 1 = 0{,}8415$; $\cos 1 = 0{,}5403$; $\sin 0{,}01 = 0{,}0100$; $\cos 0{,}01 = 0{,}99995 \approx 1{,}0000$.
3. $p = 2$: $\sin 2 = 0{,}9093$; $\cos 2 = -0{,}4161$; $\sin 0{,}02 = 0{,}0200$; $\cos 0{,}02 = 0{,}9998$.

Resultado.

$$\mathrm{PE}(1) = (0{,}8415;\ 0{,}5403;\ 0{,}0100;\ 1{,}0000), \qquad \mathrm{PE}(2) = (0{,}9093;\ -0{,}4161;\ 0{,}0200;\ 0{,}9998).$$

Comprobación. $0{,}8415^2 + 0{,}5403^2 = 0{,}7081 + 0{,}2919 = 1{,}0000$ y $0{,}9093^2 + 0{,}4161^2 = 0{,}8268 + 0{,}1731 = 0{,}9999 \approx 1$; el par lento casi no cambia entre $p = 1$ y $p = 2$.

Solución del ejercicio 4.3

Datos. $d = 2$, $\omega_0 = 1$; $\mathrm{PE}(p) = (\sin p, \cos p)$; $k = 1$.

Planteamiento. por el teorema 4.4, $M_1 = \left(\begin{smallmatrix} \cos 1 & \sin 1 \\ -\sin 1 & \cos 1 \end{smallmatrix}\right)$.

Desarrollo.

1. $M_1 = \begin{pmatrix} 0{,}5403 & 0{,}8415 \\ -0{,}8415 & 0{,}5403 \end{pmatrix}$.
2. Primera componente de $M_1\,\mathrm{PE}(1)$: $0{,}5403 \cdot 0{,}8415 + 0{,}8415 \cdot 0{,}5403 = 2 \cdot 0{,}4547 = 0{,}9093$.
3. Segunda: $-0{,}8415 \cdot 0{,}8415 + 0{,}5403 \cdot 0{,}5403 = -0{,}7081 + 0{,}2919 = -0{,}4162$.

Resultado. $M_1\,\mathrm{PE}(1) = (0{,}9093;\ -0{,}4162)$**, que coincide con** $\mathrm{PE}(2) = (0{,}9093;\ -0{,}4161)$ salvo una unidad en el cuarto decimal, debida a operar con datos redondeados.

Comprobación. la primera componente es $2 \sin 1 \cos 1 = \sin 2$ y la segunda $\cos^2 1 - \sin^2 1 = \cos 2$: las fórmulas del ángulo doble.

Solución del ejercicio 4.4

Datos. $\theta = \pi/6$; $q = (1, 2)$; $k = (3, 1)$; $m = 3$, $n = 1$; $\cos(\pi/6) = 0{,}8660$, $\sin(\pi/6) = 0{,}5$.

Planteamiento. girar cada vector, hacer el producto escalar y comparar con (4.1).

Desarrollo.

1. $3\theta = \pi/2$ y $R(\pi/2) = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$; $R(3\theta)q = (0 \cdot 1 - 1 \cdot 2;\ 1 \cdot 1 + 0 \cdot 2) = (-2;\ 1)$.
2. $R(\theta)k = (0{,}8660 \cdot 3 - 0{,}5 \cdot 1;\ 0{,}5 \cdot 3 + 0{,}8660 \cdot 1) = (2{,}098;\ 2{,}366)$.
3. Producto escalar: $(-2) \cdot 2{,}098 + 1 \cdot 2{,}366 = -4{,}196 + 2{,}366 = -1{,}830$.
4. Fórmula relativa con $\varphi = (n - m)\theta = -\pi/3$: $q \cdot k = 5$ y $q_1 k_2 - q_2 k_1 = 1 - 6 = -5$; $q^\top R(\varphi)k = 5 \cdot 0{,}5 - (-5) \cdot (-0{,}8660) = 2{,}5 - 4{,}330 = -1{,}830$.
5. Normas: $\|R(\theta)k\|^2 = 2{,}098^2 + 2{,}366^2 = 4{,}402 + 5{,}598 = 10{,}00 = 3^2 + 1^2$.

Resultado. $R(3\theta)q = (-2; 1)$**,** $R(\theta)k = (2{,}098;\ 2{,}366)$**; ambos caminos dan** $-1{,}830$ y la norma de k se conserva.

Comprobación. coincide con la fila $(m, n) = (3, 1)$ de la cuadro 4.1.

Solución del ejercicio 4.5

Datos. $R_m = \mathrm{diag}(R(m\theta_0), \ldots, R(m\theta_{d_k/2-1}))$.

Planteamiento. probar el caso de un bloque y extenderlo a la matriz por bloques.

Desarrollo.

1. Para un bloque, $R(a)^\top R(b) = R(b - a)$ por el lema 4.6: la traspuesta es la rotación de ángulo opuesto (paridad del seno y del coseno) y las rotaciones se componen sumando ángulos (fórmulas de la suma).
2. Si $A = \mathrm{diag}(A_1, \ldots, A_r)$, entonces $A^\top = \mathrm{diag}(A_1^\top, \ldots, A_r^\top)$: la entrada (i, j) de $A^\top$ es la (j, i) de A, que es nula fuera de los bloques diagonales.

3. Si $B = \mathrm{diag}(B_1, \ldots, B_r)$ con bloques del mismo tamaño, $AB = \mathrm{diag}(A_1B_1, \ldots, A_rB_r)$: en el producto por bloques, los términos $A_{ij}B_{jk}$ con $i \neq j$ o $j \neq k$ son nulos.
4. Por los pasos 1 a 3, $R_m^\top R_n = \mathrm{diag}\big(R((n-m)\theta_i)\big)_i = R_{n-m}$.
5. Por $(AB)^\top = B^\top A^\top$: $(R_m q)^\top (R_n k) = q^\top R_m^\top R_n k = q^\top R_{n-m} k$.
6. Caso $m = n$: $R_0 = \mathrm{diag}(R(0)) = I$ y se recupera $q \cdot k$, la puntuación sin posición.

Resultado. $(R_m q)^\top (R_n k) = q^\top R_{n-m} k$**:** la puntuación solo depende de $n - m$.

Comprobación. el ejercicio 4.4 lo verifica numéricamente: $-1{,}830$ por ambos caminos.

Solución del ejercicio 4.6

Datos. $X' = PX$ con P de permutación, de modo que $P^\top P = I$ y $P\mathbf{1} = \mathbf{1}$; identidades dadas para softmax y ReLU.

Planteamiento. seguir la permutación a través de cada operación de la capa.

Desarrollo.

1. $Q' = X'W_Q = PXW_Q = PQ$; del mismo modo $K' = PK$ y $V' = PV$.
2. $Q'(K')^\top = PQ(PK)^\top = PQK^\top P^\top$, porque $(PK)^\top = K^\top P^\top$.
3. $\mathrm{softmax}(PQK^\top P^\top/\sqrt{d}) = P\,\mathrm{softmax}(QK^\top/\sqrt{d})P^\top$; dividir por $\sqrt{d}$ no afecta a la permutación.
4. $H' = P\,\mathrm{softmax}(\cdot)P^\top PV = P\,\mathrm{softmax}(\cdot)V = PH$.
5. $H'W_1 + \mathbf{1}b_1 = PHW_1 + P\mathbf{1}b_1 = P(HW_1 + \mathbf{1}b_1)$, porque $P\mathbf{1} = \mathbf{1}$.
6. $\mathrm{ReLU}(P(\cdot)) = P\,\mathrm{ReLU}(\cdot)$; al multiplicar por W_2 y sumar $\mathbf{1}b_2 = P\mathbf{1}b_2$, la P queda delante: $Z' = PZ$.
7. (b) Barajar las palabras solo baraja las filas de la salida: el vector de cada palabra no depende del orden, y frases como «el perro muerde al hombre» y «el hombre muerde al perro» no se distinguen.
8. (c) Basta considerar el par $i = 0$, $(\sin t, \cos t)$. Si coincide para dos enteros $t \neq t'$, entonces $t - t' = 2\pi k$ con k entero no nulo, luego $\pi = (t - t')/(2k)$ sería racional, lo que es falso.

Resultado. $Z' = PZ$**; sin posiciones el modelo ignora el orden; dos posiciones enteras distintas nunca reciben la misma codificación sinusoidal.**

Comprobación. numéricamente, con $T = 3$, $d = 2$, pesos aleatorios y P la permutación cíclica, $f(PX)$ y $Pf(X)$ coinciden hasta la precisión de la máquina. Un caso casi repetido: $t - t' = 710$ desfasa el par 0 solo $710 - 113 \cdot 2\pi = 6{,}0 \cdot 10^{-5}$ rad, pero los pares lentos sí distinguen las dos posiciones.

Solución del ejercicio 4.7

Datos. $\mathrm{RoPE}(z, t) = ze^{it\theta}$; $\langle u, v\rangle = \mathrm{Re}(u\bar{v})$; $z_1 = 1 + 2i$, $z_2 = 3 + i$, $\theta = \pi/6$, $t_1 = 3$, $t_2 = 1$.

Planteamiento. el conjugado de $e^{i\alpha}$ es $e^{-i\alpha}$ y los exponentes se suman (proposición 4.11).

Desarrollo.

1. Miembro izquierdo: $\mathrm{Re}\big(z_1 e^{it_1\theta}\,\bar{z}_2 e^{-it_2\theta}\big) = \mathrm{Re}\big(z_1\bar{z}_2 e^{i(t_1-t_2)\theta}\big)$.
2. Miembro derecho: $\mathrm{Re}\big(z_1 e^{i(t_1-t_2)\theta}\,\bar{z}_2 e^{-i\cdot 0}\big) = \mathrm{Re}\big(z_1\bar{z}_2 e^{i(t_1-t_2)\theta}\big)$. Son la misma expresión.

3. (b) Izquierdo: $e^{3i\theta} = e^{i\pi/2} = i$ y $z_1 i = -2 + i$; $e^{i\theta} = 0{,}8660 + 0{,}5i$ y $z_2 e^{i\theta} = (2{,}598 - 0{,}5) + i(1{,}5 + 0{,}8660) = 2{,}098 + 2{,}366i$.
4. Producto escalar: $(-2) \cdot 2{,}098 + 1 \cdot 2{,}366 = -1{,}830$.
5. Derecho: $e^{2i\theta} = e^{i\pi/3} = 0{,}5 + 0{,}8660i$ y $z_1 e^{i\pi/3} = (0{,}5 - 1{,}732) + i(0{,}8660 + 1) = -1{,}232 + 1{,}866i$; con $z_2 = 3 + i$: $(-1{,}232) \cdot 3 + 1{,}866 \cdot 1 = -3{,}696 + 1{,}866 = -1{,}830$.

Resultado. **ambos miembros valen** $\mathrm{Re}(z_1 \bar{z}_2 e^{i(t_1 - t_2)\theta})$**; con los datos,** $-1{,}830$**.**

Comprobación. es el valor del ejercicio 4.4: multiplicar por $e^{it\theta}$ es la rotación $R(t\theta)$ escrita en complejos.

A.5 Tema 5

Solución del ejercicio 5.1

Datos. L capas, dimensión d, A cabezas con $d_k = d_v = d/A$, $d_{\text{ff}} = 4d$; en (d), $d = 512$ y $L = 6$.

Planteamiento. aplicar la proposición 5.1 y la proposición 5.2.

Desarrollo.

1. (a) Cada cabeza: $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d/A}$, con $3d^2/A$ parámetros; las A cabezas: $3d^2$. Más $W_O \in \mathbb{R}^{d \times d}$: $4d^2$. No depende de A.
2. (b) $W_1 \in \mathbb{R}^{d \times 4d}$ y $W_2 \in \mathbb{R}^{4d \times d}$: $8d^2$.
3. (c) Por bloque, $4d^2 + 8d^2 = 12d^2$; con L bloques, $12Ld^2$. Los sesgos ($13d$) y las normalizaciones son $O(d)$ por bloque, despreciables frente a d^2.
4. (d) $512^2 = 262\,144$; $12 \cdot 6 \cdot 262\,144 = 18\,874\,368$.

Resultado. **(a)** $4d^2$**, independiente de** A**; (b)** $8d^2$**; (c)** $12Ld^2$**; (d)** $18\,874\,368 \approx 1{,}89 \cdot 10^7$**.**

Comprobación. la fracción despreciada en (c) es $13d/(12d^2) = 13/(12d)$, que con $d = 512$ vale $0{,}0021$.

Solución del ejercicio 5.2

Datos. $d = 768$, $L = 12$, $V = 50\,257$, $n_{\text{ctx}} = 1024$; atados; posiciones aprendidas; una LayerNorm final.

Planteamiento. bloque por la proposición 5.2(ii); fuera de los bloques, embeddings (una sola vez), posiciones y normalización final.

Desarrollo.

1. $d^2 = 589\,824$; $12d^2 = 7\,077\,888$; $13d = 9984$; bloque $= 7\,087\,872$.
2. 12 bloques: $12 \cdot 7\,087\,872 = 85\,054\,464$.
3. Embedding de tokens: $50\,257 \cdot 768 = 38\,597\,376$. Posiciones: $1024 \cdot 768 = 786\,432$. LayerNorm final: $2 \cdot 768 = 1536$.
4. Total: $85\,054\,464 + 38\,597\,376 + 786\,432 + 1536 = 124\,439\,808$.
5. $12Ld^2 = 12 \cdot 12 \cdot 589\,824 = 84\,934\,656$; fracción $84\,934\,656/124\,439\,808 = 0{,}6825$.

Resultado. **bloque** $= 7\,087\,872$**;** $N = 124\,439\,808$**;** $12Ld^2$ **cubre solo el** $68{,}3\,\%$**.**

Comprobación. es el «GPT-2 de 124 M» de las implementaciones abiertas; en un modelo pequeño los embeddings pesan mucho (31 %, ejercicio 1.2).

Solución del ejercicio 5.3

Datos. $d = 5120$, $L = 40$, $d_{\text{ff}} = 13\,824$, $V = 32\,000$; sin atar; RMSNorm sin sesgo.

Planteamiento. aplicar el teorema 5.3.

Desarrollo.

1. $d^2 = 26\,214\,400$; atención $4d^2 = 104\,857\,600$.
2. SwiGLU: $5120 \cdot 13\,824 = 70\,778\,880$; por 3: $212\,336\,640$.
3. Normalizaciones: $2d = 10\,240$. Bloque: $104\,857\,600 + 212\,336\,640 + 10\,240 = 317\,204\,480$.
4. 40 bloques: $12\,688\,179\,200$.
5. Embeddings: $2 \cdot 32\,000 \cdot 5120 = 327\,680\,000$. Normalización final: 5120.
6. $N = 12\,688\,179\,200 + 327\,680\,000 + 5120 = 13\,015\,864\,320$.
7. $12Ld^2 = 12 \cdot 40 \cdot 26\,214\,400 = 12\,582\,912\,000$; cociente 0,9667.

Resultado. $N = 13\,015\,864\,320 \approx 1{,}302 \cdot 10^{10}$**;** $12Ld^2$ **cubre el** $96{,}7\,\%$**.**

Comprobación. el 3,3 % restante son los embeddings (2,5 %) y el exceso del redondeo de d_{ff} hacia arriba ($13\,824 > \frac{8}{3} \cdot 5120 = 13\,653{,}3$).

Solución del ejercicio 5.4

Datos. $N = 13\,015\,864\,320$; 2 y 16 bytes por parámetro (proposición 5.7); 1 GB $= 10^9$ bytes.

Planteamiento. memoria $= N\cdot$ bytes por parámetro.

Desarrollo.

1. Pesos en bf16: $13\,015\,864\,320 \cdot 2 = 26\,031\,728\,640$ bytes $= 26{,}03$ GB.
2. Entrenamiento: $13\,015\,864\,320 \cdot 16 = 208\,253\,829\,120$ bytes $= 208{,}3$ GB.
3. GPU de 80 GB: $208{,}3/80 = 2{,}603$, luego al menos 3.

Resultado. 26,03 **GB para la inferencia;** 208,3 **GB para el entrenamiento sin activaciones; al menos 3 GPU de 80 GB**, repartiendo el estado como en ZeRO (módulo 17).

Comprobación. $208{,}3/26{,}03 = 8{,}00 = 16/2$.

Solución del ejercicio 5.5

Datos. $V = 50\,257$, $L = 48$, $d = 1600$, $d_{\text{ff}} = 4288$; sin atar; sin sesgos; dos RMSNorm por bloque y una final; fp32 = 4 bytes.

Planteamiento. aplicar el teorema 5.3.

Desarrollo.

1. Atención: $d^2 = 2\,560\,000$; $4d^2 = 10\,240\,000$.
2. SwiGLU: $3 \cdot 1600 \cdot 4288 = 20\,582\,400$ (ejercicio 3.4).
3. Normalizaciones: 3200. Bloque: $10\,240\,000 + 20\,582\,400 + 3200 = 30\,825\,600$.

4. 48 bloques: 1 479 628 800.
5. Embeddings: $2 \cdot 50\,257 \cdot 1600 = 160\,822\,400$. Normalización final: 1600.
6. $N = 1\,479\,628\,800 + 160\,822\,400 + 1600 = 1\,640\,452\,800$.
7. Memoria en fp32: $1\,640\,452\,800 \cdot 4 = 6\,561\,811\,200$ bytes = 6,562 GB (6,111 GiB).

Resultado. $N = 1\,640\,452\,800 \approx 1{,}64 \cdot 10^9$ **parámetros;** 6,56 **GB en fp32.**

Comprobación. $12Ld^2 = 1\,474\,560\,000$ es el 89,9 % de N; la diferencia son casi exclusivamente los dos embeddings.

Solución del ejercicio 5.6

Datos. 24 GB $= 2{,}4 \cdot 10^{10}$ bytes; 2, 0,5 y 16 bytes por parámetro.

Planteamiento. $N_{\max}$ = memoria/bytes por parámetro.

Desarrollo.

1. (a) bf16: $2{,}4 \cdot 10^{10}/2 = 1{,}2 \cdot 10^{10}$ parámetros.
2. (b) int4: $2{,}4 \cdot 10^{10}/0{,}5 = 4{,}8 \cdot 10^{10}$ parámetros.
3. (c) Entrenamiento: $2{,}4 \cdot 10^{10}/16 = 1{,}5 \cdot 10^9$ parámetros.
4. Para la inferencia sobran el gradiente (2 bytes), la copia maestra en fp32 (4) y los momentos m y v (8): 14 de los 16 bytes.

Resultado. **(a)** $1{,}2 \cdot 10^{10}$**; (b)** $4{,}8 \cdot 10^{10}$**; (c)** $1{,}5 \cdot 10^9$ **parámetros.** El estado de entrenamiento ocupa 8 veces más que los pesos en bf16.

Comprobación. LLaMA-2 7B cabe en (a) (13,5 GB, cuadro 5.2) pero no en (c), que exigiría 107,8 GB.

Solución del ejercicio 5.7

Datos. $N = 12Ld^2 + 2Ld + 2Vd + d$ (corolario 5.4); $L = \alpha d$; V y α fijos.

Planteamiento. sustituir $L = \alpha d$ y quedarse con el término dominante del denominador.

Desarrollo.

1. $N = 12\alpha d^3 + 2\alpha d^2 + 2Vd + d$.
2. $f = \dfrac{2Vd}{12\alpha d^3 + 2\alpha d^2 + 2Vd + d} = \dfrac{2V}{12\alpha d^2} \cdot \dfrac{1}{1 + \frac{1}{6d} + \frac{V}{6\alpha d^2} + \frac{1}{12\alpha d^2}}$.
3. El segundo factor tiende a 1 cuando $d \to \infty$; luego $f \sim V/(6\alpha d^2)$.
4. (b) Valores exactos con los N del ejemplo 5.5 y del ejercicio 5.5. Para LLaMA-2 7B, $f = 262\,144\,000/6\,738\,415\,616 = 0{,}0389$.
5. Para el modelo del ejercicio 5.5, $f = 160\,822\,400/1\,640\,452\,800 = 0{,}0980$.
6. Aproximación asintótica: $32\,000/(6 \cdot 32 \cdot 4096) = 0{,}0407$ y $50\,257/(6 \cdot 48 \cdot 1600) = 0{,}1091$.

Resultado. $f \sim V/(6\alpha d^2)$: con un vocabulario fijo, los embeddings son una fracción despreciable de los modelos grandes (3,9 % en LLaMA-2 7B frente a 9,8 % en el modelo de $1{,}6 \cdot 10^9$ y 31 % en GPT-2 *small*). Atar ahorra poco en un modelo grande y renuncia a la flexibilidad de dos matrices distintas.

Comprobación. la aproximación sobreestima ligeramente f, porque desprecia en el denominador el propio término $2Vd$.

A.6 Tema 6

Solución del ejercicio 6.1

Datos. $y = xW$ con $x \in \mathbb{R}^2$ y $W \in \mathbb{R}^{2\times 3}$.

Planteamiento. contar las operaciones que intervienen por cada peso.

Desarrollo.

1. Cada salida es $y_j = \sum_i x_i W_{ij}$: cada peso W_{ij} interviene en una multiplicación $x_i W_{ij}$.
2. Cada producto se acumula en y_j con una suma.
3. Por token, 2 FLOPs por peso; como cada token pasa una vez por cada peso de las capas lineales, la pasada hacia delante cuesta unos $2N$ FLOPs por token (proposición 6.3).
4. Con $W \in \mathbb{R}^{2\times 3}$: 3 salidas, cada una con 2 productos y 2 sumas (contando la acumulación inicial), $3 \cdot 4 = 12$ FLOPs.

Resultado. 2 **FLOPs por parámetro y por token: una multiplicación y una suma.**

Comprobación. $12 = 2 \cdot 6$, con 6 pesos; coincide con el lema 6.2 con $m = 1$, $k = 2$ y $p = 3$.

Solución del ejercicio 6.2

Datos. $N = 6{,}738 \cdot 10^9$; $D = 2 \cdot 10^{12}$; $312 \cdot 10^{12}$ FLOP/s; aprovechamiento del 40 %; 184 320 h declaradas.

Planteamiento. $C \approx 6ND$ (corolario 6.7); tiempo $= C/(\text{aprovechamiento} \cdot \text{máximo})$.

Desarrollo.

1. $6 \cdot 6{,}738 \cdot 10^9 = 4{,}043 \cdot 10^{10}$; $C = 4{,}043 \cdot 10^{10} \cdot 2 \cdot 10^{12} = 8{,}086 \cdot 10^{22}$ FLOPs.
2. Velocidad efectiva: $0{,}4 \cdot 312 \cdot 10^{12} = 1{,}248 \cdot 10^{14}$ FLOP/s.
3. Tiempo: $8{,}086 \cdot 10^{22} / 1{,}248 \cdot 10^{14} = 6{,}479 \cdot 10^8$ s; en horas, $6{,}479 \cdot 10^8 / 3600 = 1{,}800 \cdot 10^5$ h.
4. Aprovechamiento real: $8{,}086 \cdot 10^{22} / (184\,320 \cdot 3600 \cdot 312 \cdot 10^{12}) = 8{,}086 \cdot 10^{22} / 2{,}070 \cdot 10^{23} = 0{,}3906$.

Resultado. $C \approx 8{,}09 \cdot 10^{22}$ **FLOPs; unas** 180 000 **horas de A100; aprovechamiento real del** 39 %**.**

Comprobación. $180\,000 / 184\,320 = 0{,}977$, y $0{,}3906 / 0{,}4 = 0{,}977$: las dos estimaciones son coherentes.

Solución del ejercicio 6.3

Datos. $L = 12$, $d = 768$, $d_{\text{ff}} = 2048$, $V = 50\,257$, $n = 1024$; regla $2mkp$.

Planteamiento. enumerar los productos con sus formas; las operaciones elemento a elemento se desprecian (proposición 6.3).

Desarrollo.

1. Proyecciones Q, K, V: $(n \times d)(d \times 3d)$: $2 \cdot 1024 \cdot 768 \cdot 2304 = 3\,623\,878\,656$.
2. $QK^\top$ en todas las cabezas: $2n^2d = 2 \cdot 1024^2 \cdot 768 = 1\,610\,612\,736$.
3. Pesos por V: $2n^2d = 1\,610\,612\,736$.
4. Salida W_O: $2 \cdot 1024 \cdot 768^2 = 1\,207\,959\,552$.
5. SwiGLU, tres productos $(n \times d)(d \times d_{\text{ff}})$: $3 \cdot 2 \cdot 1024 \cdot 768 \cdot 2048 = 9\,663\,676\,416$.
6. Bloque: suma de 1 a 5, $17\,716\,740\,096$. Doce bloques: $212\,600\,881\,152$.
7. Unembedding: $2 \cdot 1024 \cdot 768 \cdot 50\,257 = 79\,047\,426\,048$.
8. Total: $212\,600\,881\,152 + 79\,047\,426\,048 = 291\,648\,307\,200 \approx 2{,}916 \cdot 10^{11}$.
9. Proporciones: proyecciones de atención 19,9 %; $QK^\top$ y AV 13,3 %; MLP 39,8 %; unembedding 27,1 %.

Resultado. **pasada hacia delante $\approx 2{,}916 \cdot 10^{11}$ FLOPs por secuencia; domina el MLP (39,8 %), seguido del unembedding (27,1 %).** La comparación con el modelo con forma de GPT-2 XL ($n = 1024$) es:

Componente	GPT-2 *small*	GPT-2 XL
Proyecciones Q, K, V, O	19,9 %	28,6 %
$QK^\top$ y AV	13,3 %	9,2 %
MLP SwiGLU	39,8 %	57,5 %
Unembedding	27,1 %	4,7 %

Al crecer el modelo, el MLP y las proyecciones, que escalan con d^2, ganan peso; el unembedding, que escala con d, y la parte cuadrática, n^2d con n fijo, lo pierden.

Comprobación. sin la parte cuadrática, el coste debe ser $2nN_{\text{mat}}$ con $N_{\text{mat}} = 12 \cdot 7\,077\,888 + 38\,597\,376 = 123\,532\,032$: $2 \cdot 1024 \cdot 123\,532\,032 = 252\,993\,601\,536$, y sumando $12 \cdot 3\,221\,225\,472 = 38\,654\,705\,664$ se recupera $291\,648\,307\,200$.

Solución del ejercicio 6.4

Datos. modelo del ejercicio 5.5; n pasa de 1024 a 16 384, un factor 16.

Planteamiento. los productos con pesos escalan con n; $QK^\top$ y AV escalan con n^2.

Desarrollo.

1. Con $n = 1024$ (como en el ejercicio 6.3): bloque $= 6{,}984 \cdot 10^{10}$; unembedding $= 1{,}647 \cdot 10^{11}$; total $= 48 \cdot 6{,}984 \cdot 10^{10} + 1{,}647 \cdot 10^{11} = 3{,}517 \cdot 10^{12}$.
2. Parte cuadrática: $48 \cdot 4 \cdot 1024^2 \cdot 1600 = 3{,}221 \cdot 10^{11}$; parte lineal: $3{,}517 \cdot 10^{12} - 3{,}221 \cdot 10^{11} = 3{,}195 \cdot 10^{12}$.
3. Con $n = 16\,384$: lineal $16 \cdot 3{,}195 \cdot 10^{12} = 5{,}111 \cdot 10^{13}$; cuadrática $256 \cdot 3{,}221 \cdot 10^{11} = 8{,}246 \cdot 10^{13}$.
4. Total: $1{,}336 \cdot 10^{14}$; factor $1{,}336 \cdot 10^{14} / 3{,}517 \cdot 10^{12} = 37{,}98$.
5. Fracción de atención: $8{,}246 \cdot 10^{13} / 1{,}336 \cdot 10^{14} = 0{,}617$; MLP: 0,242.

Resultado. **el coste se multiplica por** ≈ 38 **(no por** 16**) y** $QK^\top$ **y** AV **pasan del** 9,2 % **al** 61,7 %; el MLP baja al 24,2 %.

Comprobación. el factor está entre 16 (todo lineal) y 256 (todo cuadrático): $16 \cdot 0{,}9084 + 256 \cdot 0{,}0916 = 37{,}98$.

Solución del ejercicio 6.5

Datos. pasada hacia delante por secuencia $F = 3{,}517 \cdot 10^{12}$ (ejercicio 6.4); lotes de $B = 1024$; 400 000 pasos; retroceso $= 2F$; $495 \cdot 10^{12}$ FLOP/s al 50 %; $N = 1{,}640 \cdot 10^{9}$.

Planteamiento. coste por paso $= 3F \cdot B$; tiempo = coste total / velocidad efectiva.

Desarrollo.

1. Por secuencia: $3F = 1{,}055 \cdot 10^{13}$ FLOPs.
2. Por paso: $1{,}055 \cdot 10^{13} \cdot 1024 = 1{,}080 \cdot 10^{16}$.
3. Total: $1{,}080 \cdot 10^{16} \cdot 4 \cdot 10^{5} = 4{,}321 \cdot 10^{21}$.
4. Velocidad efectiva: $0{,}5 \cdot 495 \cdot 10^{12} = 2{,}475 \cdot 10^{14}$ FLOP/s.
5. Tiempo: $4{,}321 \cdot 10^{21} / 2{,}475 \cdot 10^{14} = 1{,}746 \cdot 10^{7}$ s $= 4850$ h.
6. Regla $6ND$: $D = 4 \cdot 10^{5} \cdot 1024 \cdot 1024 = 4{,}194 \cdot 10^{11}$ tokens; $6 \cdot 1{,}640 \cdot 10^{9} \cdot 4{,}194 \cdot 10^{11} = 4{,}128 \cdot 10^{21}$ FLOPs, es decir, 4633 h.

Resultado. **unas** 4850 **h, es decir, unos** 202 **días en una sola H100.**

Comprobación. $6ND$ da un 4,5 % menos: ignora $QK^\top$ y AV y, en cambio, cuenta como multiplicados los $8{,}04 \cdot 10^{7}$ parámetros del embedding de entrada, que solo se indexan. Las dos cifras son del mismo orden.

Solución del ejercicio 6.6

Datos. $Y = XW + \mathbf{1}c^\top$, $X \in \mathbb{R}^{n \times a}$, $W \in \mathbb{R}^{a \times b}$, $c \in \mathbb{R}^{b}$, $G = \partial \mathcal{L} / \partial Y$.

Planteamiento. obtener los gradientes con la regla de la cadena (módulo 10) y contar con el lema 6.2.

Desarrollo.

1. Pasada hacia delante: XW cuesta $2nab$; sumar el sesgo a las nb entradas, nb.
2. $Y_{tj} = \sum_r X_{tr} W_{rj} + c_j$, luego $\partial \mathcal{L} / \partial X_{tr} = \sum_j G_{tj} W_{rj}$, es decir, $\partial \mathcal{L} / \partial X = G W^\top$: $2nba$ FLOPs.
3. $\partial \mathcal{L} / \partial W_{rj} = \sum_t X_{tr} G_{tj}$, es decir, $\partial \mathcal{L} / \partial W = X^\top G$: $2anb$ FLOPs.
4. $\partial \mathcal{L} / \partial c_j = \sum_t G_{tj}$, es decir, $\partial \mathcal{L} / \partial c = G^\top \mathbf{1}$: unas nb sumas.
5. Total: $6nab + 2nb$. El cociente entre el coste del sesgo y el de W es $2nb/(6nab) = 1/(3a) \to 0$ cuando a crece.
6. Sin sesgo, con $n = 1$ y $a = b = 2$: pasada hacia delante $2 \cdot 1 \cdot 2 \cdot 2 = 8$; retroceso 16; total $24 = 6 \cdot 1 \cdot 4$.

Resultado. **pasada hacia delante** $2nab$**, retroceso** $4nab$**, más** $O(nb)$ **por el sesgo**; por peso y por token, 6 FLOPs, de donde $C \approx 6ND$.

Comprobación. con $a = 768$, el sesgo supone $1/(3 \cdot 768) = 0{,}043$ % del coste de la capa.

Solución del ejercicio 6.7

Datos. $L = 32$, $d = 4096$, $n = 4096$, $N = 6\,738\,415\,616$.

Planteamiento. comparar $4Lnd$ con $2N$ (proposición 6.3 y corolario 6.5).

Desarrollo.

1. (a) $4 \cdot 32 \cdot 4096 \cdot 4096 = 128 \cdot 16\,777\,216 = 2\,147\,483\,648$; $2N = 13\,476\,831\,232$; fracción $= 0{,}1593$.
2. (b) $n^* = N/(2Ld) = 6\,738\,415\,616/262\,144 = 25\,705$.
3. (c) La fracción es $4Lnd/(2N) = n/n^*$, lineal en n; con $n = 4096$, $4096/25\,705 = 0{,}1593$.

Resultado. **con** $n = 4096$ **la atención añade un** $15{,}9\,\%$ **al coste** $2N$**; lo iguala con** $n^* \approx 25\,705$ **tokens.**

Comprobación. (a) y (c) dan el mismo valor; con contextos de 10^5 tokens la atención cuadruplicaría el coste de los pesos ($10^5/25\,705 = 3{,}9$).

Bibliografía

[1] C. M. Bishop y H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Cap. 12, «Transformers» (§12.1 «Attention», en particular §12.1.7 «Transformer layers» y §12.1.9 «Positional encoding»; §12.3.1 «Decoder transformers»). https://www.bishopbook.com/

[2] CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2025. Práctica 4, «Self-Attention, Transformers, and Pretraining». https://web.stanford.edu/class/cs224n/assignments_w25/a4.pdf

[3] Y. N. Dauphin, A. Fan, M. Auli y D. Grangier. Language modeling with gated convolutional networks. En *International Conference on Machine Learning (ICML)*, 2017. https://arxiv.org/abs/1612.08083

[4] N. Elhage *et al.* A mathematical framework for Transformer circuits. *Transformer Circuits Thread*, Anthropic, 2021. https://transformer-circuits.pub/2021/framework/index.html

[5] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Informe técnico, 2024. https://arxiv.org/abs/2403.05530

[6] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 1, «Basics», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[7] D. Hendrycks y K. Gimpel. Gaussian error linear units (GELUs). Prepublicación, 2016. https://arxiv.org/abs/1606.08415

[8] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 7, «Transformers and Pretraining». https://web.stanford.edu/~jurafsky/slp3/

[9] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. Prepublicación, 2020. https://arxiv.org/abs/2001.08361

[10] Meta. Código de inferencia de Llama 2, archivo `llama/model.py`. Repositorio `meta-llama/llama`, 2023. https://github.com/meta-llama/llama

[11] O. Press y L. Wolf. Using the output embedding to improve language models. En *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017. https://arxiv.org/abs/1608.05859

[12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[13] N. Shazeer. GLU variants improve Transformer. Prepublicación, 2020. https://arxiv.org/abs/2002.05202

[14] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen e Y. Liu. RoFormer: Enhanced Transformer with rotary position embedding. Prepublicación, 2021. Publicado en *Neurocomputing*, 568, 2024. https://arxiv.org/abs/2104.09864

[15] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[16] H. Touvron *et al.* Llama 2: Open foundation and fine-tuned chat models. Prepublicación, 2023. https://arxiv.org/abs/2307.09288

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[18] xAI. Grok-1: código y pesos publicados. 2024. https://github.com/xai-org/grok-1

[19] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang y T.-Y. Liu. On layer normalization in the Transformer architecture. En *International Conference on Machine Learning (ICML)*, 2020. https://arxiv.org/abs/2002.04745

[20] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 11, «Attention Mechanisms and Transformers» (§11.6 «Self-Attention and Positional Encoding» y §11.7 «The Transformer Architecture»). https://d2l.ai/

[21] B. Zhang y R. Sennrich. Root mean square layer normalization. En *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. https://arxiv.org/abs/1910.07467